

《人工智能引论》复习文档

《人工智能引论》第二章：知识表达与推理 ——复习文档

一、命题逻辑 (Propositional Logic)

1. 命题定义

- 命题**：确定为真或假的陈述句。
- 真值用 T (True) 或 F (False) 表示。
- 非命题例子：祈使句、疑问句、感叹句、悖论（如“我正在说谎”）。

2. 原子命题与复合命题

- 原子命题**：不包含其他命题作为组成部分的简单命题。
- 复合命题**：由原子命题通过联结词构成。

3. 命题联结词

联结词	符号	含义
与 (合取)	$p \wedge q$	“p 且 q”
或 (析取)	$p \vee q$	“p 或 q”
非 (否定)	$\neg p$	“非 p”
条件 (蕴含)	$p \rightarrow q$	“如果 p, 则 q”
双向条件 (等价)	$p \leftrightarrow q$	“p 当且仅当 q”

注：在 $p \rightarrow q$ 中, 若 $p = F$, 则无论 q 如何, 整个命题恒为真 (因为空集是任何集合的子集)。

4. 真值表

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

5. 逻辑等价 (Logical Equivalences)

常用等价公式如下：

- 交换律：

$$\alpha \wedge \beta \equiv \beta \wedge \alpha, \quad \alpha \vee \beta \equiv \beta \vee \alpha$$

- 结合律：

$$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma), \quad (\alpha \vee \beta) \vee \gamma \equiv \alpha \vee (\beta \vee \gamma)$$

- 双重否定：

$$\neg(\neg\alpha) \equiv \alpha$$

- 德·摩根定律：

$$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta, \quad \neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$$

- 蕴涵消除：

$$\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta$$

- 逆否命题：

$$\alpha \rightarrow \beta \equiv \neg\beta \rightarrow \neg\alpha$$

- 双向消除：

$$\alpha \leftrightarrow \beta \equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$$

- 分配律：

$$\begin{aligned} \alpha \wedge (\beta \vee \gamma) &\equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma) \\ \alpha \vee (\beta \wedge \gamma) &\equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma) \end{aligned}$$

6. 推理规则 (Inference Rules)

- 假言推理 (Modus Ponens) :

$$\frac{\alpha \rightarrow \beta, \alpha}{\beta}$$

- 与消解 (And-Elimination) :

$$\frac{\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n}{\alpha_i}$$

- 与导入 (And-Introduction) :

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_n}{\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n}$$

- 双重否定消去：

$$\frac{\neg\neg\alpha}{\alpha}$$

- 单项归结 (Unit Resolution) :

$$\frac{\alpha \vee \beta, \neg \beta}{\alpha}$$

- 归结 (Resolution) :

$$\frac{\alpha \vee \beta, \neg \beta \vee \gamma}{\alpha \vee \gamma}$$

二、谓词逻辑 (Predicate Logic / First-Order Logic, FOL)

1. 动机

命题逻辑无法刻画个体、属性和关系。例如苏格拉底三段论：

- 所有人都是要死的。
- 苏格拉底是人。
- 所以苏格拉底是要死的。

此推理需引入**个体**、**谓词**、**量词**。

2. 基本元素

- 个体 (Individuals)** : 具体或抽象对象 (如“苏格拉底”、“北京”)。
- 谓词 (Predicates)** :
 - 一元谓词: 描述属性, 如 $P(x)$: x 是质数。
 - 多元谓词: 描述关系, 如 $\text{Father}(x, y)$ 表示 x 是 y 的父亲。
- 函数 vs 谓词**:
 - 函数: 从定义域映射到值域。
 - 谓词: 从定义域映射到 $\{T, F\}$ 。

3. 量词 (Quantifiers)

- 全称量词**: $\forall x$, 表示“对所有 x ”。
- 存在量词**: $\exists x$, 表示“存在某个 x ”。

量词分配律 (注意成立与否) :

- $\forall x(A(x) \wedge B(x)) \equiv (\forall xA(x)) \wedge (\forall xB(x))$
- $\forall x(A(x) \vee B(x)) \not\equiv (\forall xA(x)) \vee (\forall xB(x))$
- $\exists x(A(x) \vee B(x)) \equiv (\exists xA(x)) \vee (\exists xB(x))$
- $\exists x(A(x) \wedge B(x)) \not\equiv (\exists xA(x)) \wedge (\exists xB(x))$

4. 自然语言 $\rightarrow$ 谓词逻辑

例:

- “所有国王都是人”:

$$\forall x(\text{King}(x) \rightarrow \text{Person}(x))$$

- “所有国王都头戴皇冠”:

$$\forall x(\text{King}(x) \rightarrow \text{Head_On}(\text{Crown}, x))$$

5. 量词推理规则

- 全称量词消去 (Universal Instantiation, UI) :

$$\forall x A(x) \vdash A(y)$$

- 全称量词引入 (Universal Generalization, UG) :

$$A(y) \vdash \forall x A(x) \quad (\text{要求 } y \text{ 是任意个体})$$

- 存在量词消去 (Existential Instantiation, EI) :

$$\exists x A(x) \vdash A(c) \quad (c \text{ 是新常量})$$

- 存在量词引入 (Existential Generalization, EG) :

$$A(c) \vdash \exists x A(x)$$

6. 推理示例

已知:

$$\forall x(P(x) \rightarrow Q(x)), \quad \forall x(Q(x) \rightarrow R(x))$$

证明:

$$\forall x(P(x) \rightarrow R(x))$$

证明过程:

- $\forall x(P(x) \rightarrow Q(x))$ (前提)
- $P(x) \rightarrow Q(x)$ (UI)
- $\forall x(Q(x) \rightarrow R(x))$ (前提)
- $Q(x) \rightarrow R(x)$ (UI)
- $P(x) \rightarrow R(x)$ (假言三段论)
- $\forall x(P(x) \rightarrow R(x))$ (UG)

三、知识图谱推理 (Knowledge Graph Reasoning)

1. 知识图谱定义

- 有向图结构, 节点 = 实体, 边 = 关系。
- 三元组形式: $\langle \text{head}, \text{relation}, \text{tail} \rangle$
- 可用一阶逻辑表示, 如 $\text{Couple}(\text{James}, \text{David})$

2. 归纳与演绎

- 归纳（从数据到规则）：

$$\forall x \forall y \forall z (\text{Mother}(z, y) \wedge \text{Couple}(x, z) \rightarrow \text{Father}(x, y))$$

- 演绎（从规则到新事实）：

已知 $\text{Mother}(\text{James}, \text{Ann})$ 和 $\text{Couple}(\text{James}, \text{David})$, 可推出 $\text{Father}(\text{David}, \text{Ann})$

3. 归纳逻辑程序设计 (ILP)

- 结合机器学习与逻辑推理。
- 目标：从正例、反例、背景知识中学习一阶逻辑规则。

4. FOIL 算法 (First Order Inductive Learner)

输入：

- 目标谓词 P
- 正例集 E^+ , 反例集 E^-
- 背景知识（其他谓词实例）

输出：

- 推理规则：前提 $\rightarrow$ 目标谓词

算法步骤：

1. 初始化规则：目标谓词（无前提）。
2. 逐个尝试添加前提约束谓词（来自背景知识）。
3. 计算 **FOIL 信息增益**：

$$\text{FOIL_Gain} = m' \left(\log_2 \frac{m'}{m' + n'} - \log_2 \frac{m}{m + n} \right)$$

其中：

- m, n : 原规则覆盖的正例、反例数
- m', n' : 新规则覆盖的正例、反例数

4. 选择增益最大的前提加入规则。
5. 移除被新规则排除的训练样例。
6. 重复直到无反例被覆盖。

示例：

目标谓词： $\text{Father}(x, y)$

学习得规则：

$$\forall x \forall y \forall z (\text{Mother}(z, y) \wedge \text{Couple}(x, z) \rightarrow \text{Father}(x, y))$$

5. 路径排序算法 (Path Ranking Algorithm, PRA)

步骤:

- 特征抽取:** 生成实体间的关系路径 (如 Couple $\rightarrow$ Mother)
- 特征计算:** 对每个训练对 (s, t) , 判断是否可通过某路径连接 (0/1 或概率)
- 分类器训练:** 用特征向量训练分类器 (如逻辑回归)
- 预测:** 对新实体对, 提取路径特征, 输入分类器判断关系是否存在

示例:

- 正例: $(David, Mike) \rightarrow$ 路径: Couple $\rightarrow$ Mother
- 特征向量: $[1, 0, 0, 0]$
- 分类器输出 $1 \Rightarrow \text{Father}(David, Mike)$ 成立

四、概率图推理 (Probabilistic Graphical Models)

1. 动机

现实世界存在不确定性, 需用概率建模依赖关系。

2. 两类模型

- 贝叶斯网络 (Bayesian Network):** 有向无环图 (DAG), 表示因果依赖。
- 马尔可夫网络 (Markov Network):** 无向图, 表示对称依赖。

3. 贝叶斯网络性质

- 局部马尔可夫性:** 给定父节点, 节点独立于非后代。
- 联合分布分解:**

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{Pa}(x_i))$$

其中 $\text{Pa}(x_i)$ 是 x_i 的父节点集合。

示例:

$$P(\text{Cloudy}, \text{Rain}, \text{Sprinkler}, \text{WetGrass}) = P(C)P(S|C)P(R|C)P(W|S, R)$$

4. 马尔可夫逻辑网络 (Markov Logic Networks, MLNs)

将一阶逻辑规则赋予权重, 构建概率模型。

概率公式:

$$P(X = x) = \frac{1}{Z} \exp \left(\sum_i w_i n_i(x) \right)$$

- w_i : 第 i 条规则的权重
- $n_i(x)$: 在状态 x 下第 i 条规则成立的次数 (0 或 1)

- Z : 归一化常数 (partition function)

示例规则:

规则	描述	权重
$r_1 : \text{indoor} \vee \text{outdoor}$	必须选其一	1
$r_2 : \neg(\text{indoor} \wedge \text{outdoor})$	不能同时	9
$r_3 : \text{rain} \rightarrow \text{indoor}$	下雨则室内	5
$r_4 : \text{lightshow} \rightarrow \text{outdoor}$	灯光秀则室外	7

计算:

$$P(\text{outdoor} \wedge \neg \text{indoor}) = \frac{1}{Z} \exp(1 + 9 + 0 + 7) = \frac{e^{17}}{Z}$$

五、因果推理 (Causal Reasoning)

1. 因果 vs 关联

- 因果**: X 引起 Y
- 混淆偏差 (Confounding Bias)**: $X \leftarrow Z \rightarrow Y$, 忽略 Z 会导致虚假关联
- 选择偏差 (Selection Bias)**: $X \rightarrow S \leftarrow Y$, 基于 S 选择样本导致虚假关联

2. 辛普森悖论 (Simpson's Paradox)

- 整体数据: 用药恢复率 < 不用药
- 按性别分组: 用药恢复率 > 不用药
- 原因: 性别是混淆变量

3. 因果理论框架

- 潜在结果框架 (Neyman-Rubin)**
- 结构因果模型 (SCM)**: 用**因果图** (DAG) 表示变量生成机制

4. do 算子 (Intervention)

- $P(Y = y \mid X = x)$: 观测概率
- $P(Y = y \mid \text{do}(X = x))$: 干预概率 (固定 $X = x$, 切断其父节点影响)

5. 调整公式 (Adjustment Formula)

若 Z 是 X 的父节点 (或满足后门准则), 则:

$$P(Y = y \mid \text{do}(X = x)) = \sum_z P(Y = y \mid X = x, Z = z)P(Z = z)$$

应用于辛普森悖论：

令 X =用药, Y =恢复, Z =性别
则：

$$P(Y = 1 \mid \text{do}(X = 1)) = P(Y = 1 \mid X = 1, Z = 1)P(Z = 1) + P(Y = 1 \mid X = 1, Z = 0)P(Z = 0)$$

代入数据得：

$$= 0.93 \times \frac{357}{700} + 0.73 \times \frac{343}{700} \approx 0.832$$

同理：

$$P(Y = 1 \mid \text{do}(X = 0)) \approx 0.7818$$

平均因果效应 (ACE)：

$$\text{ACE} = 0.832 - 0.7818 = 0.0502 > 0 \Rightarrow \text{药物有效}$$

6. 因果层次 (Pearl’s Ladder of Causation)

层次	问题类型	表达式	能力
1. 关联	What is?	$P(y \mid x)$	统计相关
2. 干预	What if?	$P(y \mid \text{do}(x))$	因果推断
3. 反事实	Why?	$P(y_x \mid x', y')$	反事实推理

六、总结

- 符号主义 AI 起源于逻辑推理。
- 命题逻辑 → 谓词逻辑：增强表达能力（个体、关系、量词）。
- 知识图谱：结构化知识，支持归纳（ILP/FOIL）与演绎推理。
- 概率图模型：处理不确定性（贝叶斯网络、MLN）。
- 因果推理：超越关联，回答“为什么”和“怎么办”。
- 推理即计算：从莱布尼茨到 Pearl，逻辑推理可形式化为计算过程。

“让我们来计算一下（Calculus）！”—— 莱布尼茨

第三章 搜索探寻与问题求解 —— 复习文档

一、搜索基本概念

1. 状态 (State)

- 定义：**对搜索算法和环境当前情形的描述。
- 简化原则：**剔除与问题无关的信息，仅保留对求解有作用的信息。
- 示例：**在图3.1中，每个城市是一个状态。

2. 动作 (Action)

- 定义：**从一个状态转移到另一个状态所采取的行为。
- 性质：**动作是离散的（非连续）。

3. 状态转移 (State Transition)

- 定义：**执行某个动作后，状态发生变化的过程。
- 示例：**从城市 A 乘坐列车到 D，即发生状态转移 $A \rightarrow D$ 。

4. 路径 (Path) 与代价 (Cost)

- 路径：**一系列状态组成的序列，如 $A \rightarrow E \rightarrow G \rightarrow K$ 。
- 单步代价：**相邻状态之间边上的数值（假设为非负）。
- 路径总代价：**路径上所有单步代价之和。

5. 目标测试 (Goal Test)

- 函数：**`goal_test(s)`
- 作用：**判断状态 s 是否为目标状态（如是否到达城市 K）。
- 注意：**目标测试通过 $\neq$ 找到最优解。

6. 搜索树 (Search Tree)

- 结构：**
 - 根节点 = 初始状态；
 - 每个节点代表一条从根出发的路径；
 - 同一状态可在不同路径下多次出现（如图3.2中三个 A）。

7. 搜索算法性能指标

指标	定义
完备性	若问题有解，算法是否一定能找到解
最优性	算法是否能找到第一个即为最优的解
时间复杂度	扩展的节点数量
空间复杂度	同时存储的节点数量

常用符号

符号	含义
b	分支因子（每个节点最多子节点数）
d	最浅目标节点深度
m	搜索树最大可能深度
n	状态空间大小

二、贪婪最佳优先搜索与 A* 搜索

1. 启发式搜索（Informed / Heuristic Search）

- 辅助信息：问题之外但相关的知识（如直线距离）。

2. 关键函数

- 启发函数 $h(n)$ ：估计从节点 n 到目标的最小代价。
- 评价函数 $f(n)$ ：用于选择下一个扩展节点。

3. 贪婪最佳优先搜索（Greedy Best-First Search）

- 评价函数：

$$f(n) = h(n)$$

- 特点：
 - 完备（若剪枝避免环路）；
 - 非最优（如图3.4中找到 $A \rightarrow E \rightarrow H \rightarrow J \rightarrow K$ ，但最短路径为 $A \rightarrow E \rightarrow G \rightarrow K$ ）；
 - 最坏时间/空间复杂度： $O(b^m)$ 。

4. A* 搜索算法

- 引入实际代价函数 $g(n)$ ：从初始节点到 n 的实际路径代价。
- 评价函数：

$$f(n) = g(n) + h(n)$$

- 示例（图3.5）：
 - $f(B) = 5 + 10 = 15$
 - $f(E) = 3 + 7 = 10 \rightarrow$ 优先扩展 E

5. 启发函数性质

(1) 可容性 (Admissibility)

- **定义**: 对任意节点 n , 有

$$h(n) \leq h^*(n)$$

其中 $h^*(n)$ 是从 n 到目标的**真实最小代价**。

- **意义**: 启发函数**不会高估**实际代价。
- **结论**: 若 $h(n)$ 可容, 则 A* 算法**最优**。

(2) 一致性 (Consistency, 又称单调性)

- **定义**: 对任意节点 n 及其后继 n' , 有

$$h(n) \leq c(n, a, n') + h(n')$$

其中 $c(n, a, n')$ 是从 n 经动作 a 到 n' 的单步代价。

- **几何解释**: 满足三角不等式 (如欧氏距离)。
- **性质**: 一致性 $\Rightarrow$ 可容性 (更强条件)。

6. A* 算法性能

- **完备性条件**:
 - 分支有限;
 - 单步代价有正下界;
 - 启发函数有下界。
- **最优性**: 若 $h(n)$ 可容, 则 A* 找到的第一个解是最优解。
- **证明思路**:
 - 设找到目标节点 n , 则 $f(n) = g(n)$;
 - 对任意边缘节点 n' , 有 $f(n) \leq f(n') = g(n') + h(n') \leq g(n') + h^*(n')$;
 - 故 $g(n) \leq$ 任何经 n' 的路径代价 $\Rightarrow$ 最优。

三、Minimax 搜索与 Alpha-Beta 剪枝

1. 对抗搜索 (Adversarial Search)

- **场景**: 二人零和博弈 (一方最大化得分, 另一方最小化)。
- **角色**:
 - **MAX**: 最大化终局得分;
 - **MIN**: 最小化终局得分。

2. 形式化定义

元素	描述
状态 s	当前局面 + 当前行动玩家
动作 $a \in \text{actions}(s)$	当前玩家可选操作
状态转移 $\text{result}(s, a)$	执行动作后的新状态
终局检测 $\text{terminal_test}(s)$	游戏是否结束
终局得分 $\text{utility}(s)$	终局状态下 MAX 的得分 (MIN 得分为 $-\text{utility}(s)$)

3. Minimax 算法

- 递归定义:

$$\text{minimax}(s) = \begin{cases} \text{utility}(s), & \text{if } \text{terminal_test}(s) \\ \max_{a \in \text{actions}(s)} \text{minimax}(\text{result}(s, a)), & \text{if } \text{player}(s) = \text{MAX} \\ \min_{a \in \text{actions}(s)} \text{minimax}(\text{result}(s, a)), & \text{if } \text{player}(s) = \text{MIN} \end{cases}$$

- 算法流程 (Algorithm 3.2) :

```
def MinimaxDecision(s):
    return argmax_{a ∈ actions(s)} MinValue(result(s, a))

def MaxValue(s):
    if terminal_test(s): return utility(s)
    v = -∞
    for a in actions(s):
        v = max(v, MinValue(result(s, a)))
    return v

def MinValue(s):
    if terminal_test(s): return utility(s)
    v = +∞
    for a in actions(s):
        v = min(v, MaxValue(result(s, a)))
    return v
```

- 复杂度:
 - 时间: $O(b^m)$
 - 空间: $O(bm)$ (深度优先)

4. Alpha-Beta 剪枝

- 思想: 在不改变 Minimax 结果的前提下, 剪去不影响决策的子树。
- 关键变量:
 - α : MAX 层当前已知的最大下界;
 - β : MIN 层当前已知的最小上界。

剪枝规则

- **Alpha 剪枝** (对 MIN 节点后代) :
 - 若某 MIN 节点已知可返回 $\leq \alpha$ 的值, 则其兄弟节点若能返回更小值, 无需继续探索。
- **Beta 剪枝** (对 MAX 节点后代) :
 - 若某 MAX 节点已知可返回 $\geq \beta$ 的值, 则其兄弟节点若能返回更大值, 无需继续探索。
- **剪枝条件**: 当 $\alpha \geq \beta$ 时, 剪枝发生。
- **效果**:
 - 最优情况下, 时间复杂度降至 $O(b^{m/2})$;
 - 相当于搜索深度翻倍。

四、蒙特卡洛树搜索 (MCTS)

1. 背景: 多臂赌博机 (Multi-Armed Bandit)

- **目标**: 在 T 次尝试中最大化总奖励。
- **挑战**: 探索 (Exploration) vs 利用 (Exploitation) 的权衡。

2. 悔值 (Regret) 函数

- **定义**:

$$R_T = Tu^* - \sum_{t=1}^T r_t$$

其中 $u^* = \max_i u_i$, r_t 为第 t 次奖励。

- **目标**: 最小化 $\mathbb{E}[R_T]$ 。

3. 策略对比

(1) 贪心算法 (Greedy)

- 总是选择当前平均奖励最高的臂。
- **缺点**: 易陷入局部最优, 忽略未充分探索的臂。

(2) ϵ -贪心

- 以概率 $1 - \epsilon$ 利用, 以 ϵ 随机探索。
- 改善探索, 但效率不高。

(3) 上限置信区间 (UCB1)

- **核心思想**: 优先选择置信上界高的动作。
- **UCB1 公式**:

$$a_t = \arg \max_i \left(\bar{x}_{i,T(i,t-1)} + C \sqrt{\frac{2 \ln t}{T(i,t-1)}} \right)$$

其中:

- $\bar{x}_{i,T}$: 动作 i 在 T 次尝试中的平均奖励;
- $T(i, t-1)$: 截至 $t-1$ 步, 动作 i 被选次数;
- C : 探索权重超参数。
- **理论保证**: 悔值期望上界为 $O\left(\frac{K \ln T}{\Delta}\right)$, 其中 $\Delta = \min_{u_i < u^*} (u^* - u_i)$ 。

4. MCTS 四步骤 (图3.20)

(1) 选择 (Selection)

- 从根开始, 用 UCB1 递归选择子节点, 直到叶子或未完全扩展节点 L 。

(2) 扩展 (Expansion)

- 若 L 非终局, 随机添加一个未扩展子节点 M 。

(3) 模拟 (Simulation)

- 从 M 出发, 用简单策略 (如随机) 模拟至终局, 得奖励 r 。

(4) 反向传播 (Backpropagation)

- 将 r 沿路径回传, 更新每个节点的:
 - 访问次数 N ;
 - 总奖励 Q (对抗搜索中, MAX/MIN 层符号相反)。

5. UCT 算法 (Algorithm 3.4)

```
def UCTsearch(s0):
    v0 = create_node(s0)
    while not max_iterations:
        v1 = SelectPolicy(v0)
        sT = SimulatePolicy(v1.state)
        BackPropagate(v1, sT)
    return argmax_{v' ∈ children(v0)} (v'.Q / v'.N)

def SelectPolicy(v):
    while not terminal_test(v.state):
        if v has unexpanded child:
            return Expand(v)
        else:
            v = UCB1(v, C)
    return v

def UCB1(v, C):
    return argmax_{v' ∈ children(v)} (v'.Q / v'.N + C * sqrt(2 * ln(v.N) / v'.N))
```

6. 对抗搜索中的 MCTS 特殊处理

- **反向传播时**:
 - MAX 层: $Q \leftarrow Q - \text{utility}(s_T)$
 - MIN 层: $Q \leftarrow Q + \text{utility}(s_T)$

- **目的**：统一使用 UCB1（始终“最大化”），通过符号转换实现 MIN 的“最小化”。

五、延伸阅读要点

1. 组合爆炸 (Combinatorial Explosion)

- **香农数**：国际象棋可能局面数约 10^{120} ，远超宇宙原子数 (10^{82})。

2. 地平线问题 (Horizon Problem)

- **定义**：搜索深度有限，无法预见“地平线”后的关键变化。
- **案例**：AlphaGo 未能应对李世石第78手“神之一手”。

3. 剪枝思想

- **Alpha-Beta 剪枝**：由 John McCarthy 提出，用于深蓝 (Deep Blue)。

六、本章小结

- 搜索是从解空间中寻找满足约束路径的过程。
- **无信息搜索**（如 BFS、DFS）vs **有信息搜索**（如 A*）。
- **对抗搜索**需考虑对手最优反应（Minimax + Alpha-Beta）。
- **MCTS** 通过采样+UCB 平衡探索与利用，适用于巨大状态空间。
- **搜索 + 学习**：现代 AI 趋势（如 AlphaGo = MCTS + 神经网络）。

注：所有公式、算法、图示说明均基于课件原文整理。

《人工智能引论》第四章：机器学习 复习文档

一、基本概念

1. 机器学习定义

- **数据驱动学习 (data-driven learning)**：从数据出发学习其中蕴含的模式并进行抽象。
- **Arthur Samuel (1959) 定义**：

“让机器具有不需要明确编程而具有的一种学习能力” (the ability to learn without being explicitly programmed)。

2. 学习范式分类

- 监督学习 (Supervised Learning)** : 训练集 $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, 学习映射 $f: \mathbf{x} \mapsto y$ 。
- 无监督学习 (Unsupervised Learning)** : 仅使用无标签数据 $\{\mathbf{x}_i\}_{i=1}^n$ 。
- 半监督学习 (Semi-supervised Learning)** : 部分有标签、部分无标签。

3. 数据划分

- 训练集 (Training Set)** : 用于模型参数优化。
- 验证集 (Validation Set)** : 用于超参选择与模型评估。
- 测试集 (Test Set)** : 用于最终性能评估。
三者互不重叠。

4. 没有免费午餐定理 (No Free Lunch Theorem)

任何两个算法在所有可能问题上的平均性能相同。
→ 必须结合具体任务引入先验知识提升性能。

5. 泛化能力 (Generalization)

- 经验风险 (Empirical Risk)** :

$$R_{\text{emp}}(f) = \frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i, f(\mathbf{x}_i))$$

- 期望风险 (Expected Risk)** :

$$R(f) = \iint \text{Loss}(y, f(\mathbf{x})) P(\mathbf{x}, y) d\mathbf{x} dy$$

- 泛化误差界:**

$$R(f) \leq R_{\text{emp}}(f) + \text{err}$$

其中 err 与模型复杂度和样本量相关。

6. 过拟合与欠拟合

经验风险	期望风险	模型状态
小	小	泛化能力强
小	大	过拟合 (Overfitting)
大	大	欠拟合 (Underfitting)

7. 结构风险最小化 (Structural Risk Minimization)

$$\min_f \left[\frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i, f(\mathbf{x}_i)) + \lambda J(f) \right]$$

- $J(f)$: 正则化项 (惩罚项)
- λ : 正则化强度系数

- 哲学基础：奥卡姆剃刀 (Occam's Razor)

二、监督学习

1. 回归分析 (Regression Analysis)

(1) 一元线性回归

- 模型形式: $y = ax + b$
- 损失函数 (最小二乘法):

$$L(a, b) = \sum_{i=1}^n (y_i - ax_i - b)^2$$

- 解析解:

$$a = \frac{\sum x_i y_i - n \bar{x} \bar{y}}{\sum x_i^2 - n \bar{x}^2}, \quad b = \bar{y} - a \bar{x}$$

(2) 多元线性回归

- 模型形式: $f(\mathbf{x}) = a_0 + \sum_{j=1}^D a_j x_j = \mathbf{a}^\top \tilde{\mathbf{x}}$, 其中 $\tilde{\mathbf{x}} = [1, x_1, \dots, x_D]^\top$
- 矩阵表示:

$$\mathbf{y} = \mathbf{X} \mathbf{a}$$

- 闭式解 (Normal Equation):

$$\mathbf{a} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

(3) 逻辑斯蒂回归 (Logistic Regression)

- 用于二分类, 输出概率:

$$y = \sigma(z) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}, \quad y \in (0, 1)$$

- 推广至多类: **Softmax 回归**

2. 决策树 (Decision Tree)

(1) 构建思想

- 树形结构, 逐层基于特征划分样本。
- 目标: 提高节点“纯度” (purity)。

(2) 信息熵 (Entropy)

- 对样本集 D , 类别 $k = 1, \dots, K$, 概率 p_k :

$$\text{Ent}(D) = - \sum_{k=1}^K p_k \log_2 p_k$$

(3) 信息增益 (Information Gain)

- 使用属性 A 划分后:

$$\text{Gain}(D, A) = \text{Ent}(D) - \sum_{i=1}^v \frac{|D_i|}{|D|} \text{Ent}(D_i)$$

其中 v 为属性取值数, D_i 为第 i 个子集。

(4) 停止条件

- 所有样本同类
- 无可分属性
- 子集样本数过小

三、无监督学习: K-Means 聚类

算法 4.1: K-Means 聚类

输入: 数据 $\{\mathbf{x}_i\}_{i=1}^n \subset \mathbb{R}^d$, 聚类数 K

输出: 每个样本的簇标签

步骤:

- 初始化:** 随机选择 K 个质心 $\{\mathbf{c}_j\}_{j=1}^K$
- 分配:** 对每个 $\mathbf{x}_i$, 分配到最近质心:

$$\text{label}(i) = \arg \min_j \|\mathbf{x}_i - \mathbf{c}_j\|_2$$

- 更新质心:**

$$\mathbf{c}_j = \frac{1}{|G_j|} \sum_{\mathbf{x}_i \in G_j} \mathbf{x}_i$$

- 迭代:** 重复 2-3 直至收敛 (质心不变或达最大迭代次数)

优化目标

$$\min_{\{G_j\}} \sum_{j=1}^K \sum_{\mathbf{x} \in G_j} \|\mathbf{x} - \mathbf{c}_j\|^2 = \min_{j=1}^K |G_j| \cdot \text{Var}(G_j)$$

注: K-Means 易受初始值影响, 通常多次运行取最优。

四、特征降维

1. 线性判别分析 (LDA, Fisher Discriminant)

适用: 监督降维 (利用类别标签)

优化目标（二分类）

- 投影方向 $\mathbf{w}$ ，使类间距离大、类内方差小：

$$J(\mathbf{w}) = \frac{\|\mathbf{w}^\top(\mathbf{m}_2 - \mathbf{m}_1)\|^2}{\mathbf{w}^\top(\mathbf{S}_1 + \mathbf{S}_2)\mathbf{w}} = \frac{\mathbf{w}^\top \mathbf{S}_b \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_w \mathbf{w}}$$

- 类间散度矩阵： $\mathbf{S}_b = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top$
- 类内散度矩阵： $\mathbf{S}_w = \sum_{i=1}^2 \sum_{\mathbf{x} \in C_i} (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^\top$

解

- 最优投影方向：

$$\mathbf{w} \propto \mathbf{S}_w^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$$

- 降维后维度上限： $\min(K - 1, d)$

2. 主成分分析 (PCA)

适用：无监督降维（保留最大方差）

数学基础

- 方差：

$$\text{Var}(X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2$$

- 协方差：

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu_X)(y_i - \mu_Y)$$

- 皮尔逊相关系数：

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}, \quad |\rho_{XY}| \leq 1$$

PCA 优化目标

- 给定中心化数据矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$
- 协方差矩阵： $\mathbf{\Sigma} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$
- 寻找投影矩阵 $\mathbf{W} \in \mathbb{R}^{d \times l}$ ，最大化投影后方差：

$$\max_{\mathbf{W}} \text{tr}(\mathbf{W}^\top \mathbf{\Sigma} \mathbf{W}) \quad \text{s.t. } \mathbf{W}^\top \mathbf{W} = \mathbf{I}$$

解

- $\mathbf{W}$ 由 $\mathbf{\Sigma}$ 的前 l 个最大特征值对应的特征向量组成。

算法 4.2: PCA

1. 中心化: $\mathbf{x}_i \leftarrow \mathbf{x}_i - \bar{\mathbf{x}}$
2. 计算协方差矩阵: $\Sigma = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$
3. 特征分解: $\Sigma = \mathbf{V} \Lambda \mathbf{V}^\top$
4. 取前 l 列 $\mathbf{W} = [\mathbf{v}_1, \dots, \mathbf{v}_l]$

3. 特征人脸 (Eigenfaces)

- 将每张 $d \times d$ 人脸图像展平为 d^2 维向量
- 应用 PCA 得到主成分 (特征向量) $\rightarrow$ “特征人脸”
- 高效实现: 使用 **奇异值分解 (SVD)**

$$\mathbf{X} = \mathbf{U} \mathbf{D} \mathbf{V}^\top$$

- 当 $d^2 \gg n$, 直接对 $\mathbf{X} \mathbf{X}^\top$ ($n \times n$) 做特征分解更高效
- 右奇异向量 $\mathbf{V}$ 即为特征人脸

五、演化学习 (Evolutionary Learning)

1. 演化算法 (Evolutionary Algorithms)

- 受自然演化启发: **突变重组 + 自然选择**
- 包括: 遗传算法 (GA)、遗传规划 (GP)、演化策略等

2. 遗传算法 (Genetic Algorithm, GA)

核心操作:

- **编码**: 候选解 $\rightarrow$ 染色体 (如浮点向量)
- **适应度函数**: 评价个体优劣
- **选择 (Selection)**: 如轮盘赌
- **交叉 (Crossover)**: 交换基因片段
- **变异 (Mutation)**: 随机扰动基因

算法流程

1. 初始化种群 (随机生成染色体)
2. 计算适应度, 记录最优个体 Best
3. **选择**: 按适应度概率复制个体
4. **交叉**: 以概率 p_c 对配对个体交换部分基因
5. **变异**: 以概率 p_m 随机改变某些基因
6. 更新 Best
7. 若未达终止条件 (最大代数/精度), 返回步骤 3

示例: 最大化函数 $f(x_1, x_2, x_3, x_4) = \frac{1}{x_1^2 + x_2^2 + x_3^2 + x_4^2 + 8}$

六、模型评估指标（二分类）

设：

- TP：真正例，FP：假正例
- TN：真反例，FN：假反例

指标	公式
准确率 (Accuracy)	$ACC = \frac{TP + TN}{TP + FP + TN + FN}$
错误率 (Error Rate)	$1 - ACC$
精确率 (Precision)	$P = \frac{TP}{TP + FP}$
召回率 (Recall)	$R = \frac{TP}{TP + FN}$
F1-score	$F_1 = \frac{2PR}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$

注意：类别不平衡时，ACC 不可靠，应优先看 Precision/Recall/F1。

七、频率学派 vs 贝叶斯学派

	频率学派	贝叶斯学派
参数观点	固定但未知	随机变量，服从先验分布
优化目标	最大似然估计 (MLE)	最大后验估计 (MAP)
公式	$\hat{\theta} = \arg \max_{\theta} P(D \theta)$	$\hat{\theta} = \arg \max_{\theta} P(\theta D) \propto P(D \theta)P(\theta)$
典型方法	MLE、置信区间	先验 + 似然 → 后验

八、损失函数（表 4.1）

名称	定义
0-1 损失	$Loss(y, f(\mathbf{x})) = \begin{cases} 0, & f(\mathbf{x}) = y \\ 1, & f(\mathbf{x}) \neq y \end{cases}$
平方损失	$(y - f(\mathbf{x}))^2$
绝对损失	$ y - f(\mathbf{x}) $
对数损失 (负对数似然)	$-\log P(y \mathbf{x})$

九、延伸：德国坦克问题（贝叶斯推断示例）

- 观察到 n 辆坦克，最大编号为 x

- 假设总数 N 为随机变量
- 后验概率：

$$P(N | x, n) \propto \frac{\binom{x-1}{n-1}}{\binom{N}{n}}, \quad N \geq x$$

十、本章小结要点

- 机器学习始于 **Samuel 的跳棋程序 (1959)**
- 监督学习构建 **从特征到语义的映射**
- 泛化能力** 是核心目标，需平衡经验风险与模型复杂度
- 先验知识** 对提升性能至关重要 (NFL 定理启示)
- 未来方向：**可学习性 (Learnability)**、**知识与数据融合**

文档依据：高等教育出版社《人工智能引论》第四章课件（吴飞，浙江大学）

已收到您上传的 PDF 文件《计算机“101计划”〈人工智能引论〉教材课件-第五章 神经网络与深度学习》。

我将根据该文件内容，**提取所有专有名词与知识点，并整理成一份详尽的复习文档**，包括：

- 所有关键概念、模型、算法；
- 所有数学公式（使用 LaTeX 格式）；
- 模型结构描述、训练机制、优缺点分析；
- 以 Markdown 格式组织，便于阅读与复习。

第五章：神经网络与深度学习 — 复习文档

基于《人工智能引论》（吴飞，浙江大学）第五章课件内容整理

一、人工神经网络概述

1.1 历史发展脉络

年份	贡献者	成果	意义
1943	Warren McCulloch & Walter Pitts	提出 MCP 神经元模型	首次用数学模型模拟生物神经元，“在科学史上第一次，我们知道了我们是怎么知道的”
1949	Donald Hebb	赫布理论 (Hebbian Theory) ： “Neurons that fire together, wire together.”	学习和记忆的本质是突触连接权重随经验刺激而改变

年份	贡献者	成果	意义
1958	Frank Rosenblatt	感知机 (Perceptron)	首个可学习的线性分类器，被誉为“能自学的电子大脑”
1974 / 1986	Werbos; Rumelhart & Hinton	误差反向传播算法 (Backpropagation)	解决多层网络参数优化问题，实现数据驱动自动学习
2006	Geoffrey Hinton et al.	深度信念网络 (Deep Belief Network, DBN)	开启现代深度学习时代，实现端到端 (end-to-end) 训练

1.2 MCP 神经元模型

- 输入： n 个二值输入 $x_i \in \{0, 1\}$
- 权重：预设连接权重 $w_i \in \{-1, 0, 1\}$
- 输出：

$$y = \phi\left(\sum_{i=1}^n w_i x_i\right), \quad \phi(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

功能：实现二分类；是人工神经网络的基本计算单元。

1.3 感知机 (Perceptron)

- 输入：实值向量 $\mathbf{x} = (x_1, \dots, x_n)$
- 偏置项： b
- 加权和： $z = \sum_{i=1}^n w_i x_i + b$
- 激活函数（阈值函数）：

$$\phi(z) = \begin{cases} 1, & z \geq 0 \\ -1, & z < 0 \end{cases}$$

局限性

- 可模拟 **AND**、**OR**、**NAND**（线性可分）
- 无法解决 **XOR 问题**（非线性可分）

引出多层结构必要性。

二、前馈神经网络 (Feedforward Neural Network)

2.1 结构特点

- 由 **输入层**、若干**隐藏层**、**输出层** 构成
- 全连接 (Fully Connected)**：相邻层神经元两两相连，同层无连接
- 前馈**：信息单向流动，无环

- **逐层抽象**: 从像素空间 → 语义空间 (如“汽车”概率)

2.2 激活函数 (非线性映射)

(1) Sigmoid 函数

$$f(x) = \frac{1}{1 + e^{-x}}, \quad f'(x) = f(x)(1 - f(x))$$

- 优点: 输出为 (0,1), 可解释为概率; 平滑可导
- 缺点: **梯度消失 (Vanishing Gradient)**, 尤其在深层网络中

(2) ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}, \quad f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

- 优点: 缓解梯度消失; 计算简单; 引入稀疏性 (防过拟合)
- 缺点: **神经元死亡 (Dead Neuron)** (负区梯度恒为0)

改进: Leaky ReLU、Parametric ReLU (PReLU)

(3) Softmax 函数 (多分类输出)

对 k 类分类, 输入 $\mathbf{x} = (x_1, \dots, x_k)$, 输出第 i 类概率:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}, \quad \text{满足 } \sum_{i=1}^k y_i = 1, \quad 0 < y_i < 1$$

用于输出层, 配合交叉熵损失。

2.3 损失函数 (Loss Function)

(1) 均方误差 (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

(2) 交叉熵损失 (Cross Entropy)

对真实标签 y_i (one-hot) 与预测概率 $\hat{y}_i$:

$$H(y, \hat{y}) = - \sum_i y_i \log \hat{y}_i$$

示例: 若真实类别为第3类 ($y = [0, 0, 1]$), 则损失为 $-\log(\hat{y}_3)$

- **性质**: 预测分布越接近真实分布, 交叉熵越小
- **优势**: 比 MSE 更适合分类任务, 梯度更稳定

2.4 参数优化：梯度下降与反向传播

梯度下降基本思想

- 目标：最小化损失函数 $L(\theta)$
- 更新规则：

$$\theta_{\text{new}} = \theta - \eta \nabla_{\theta} L(\theta)$$

其中 η 为 **学习率 (learning rate)**

三种梯度下降变体

类型	描述	优缺点
批量梯度下降 (Batch GD)	使用全部训练样本计算梯度	稳定但内存/计算开销大
随机梯度下降 (SGD)	每次用一个样本更新	波动大，收敛慢，但可跳出局部最优
小批量梯度下降 (Mini-batch GD)	每次用一小批 (如 32/64) 样本	最常用 ：兼顾效率与稳定性，支持向量化加速

误差反向传播 (Backpropagation)

- 利用 **链式法则 (Chain Rule)** 计算梯度
- 以单神经元为例 (图5.6)：

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial x} \cdot \frac{\partial x}{\partial w_1}$$

其中：

- $x = \sum w_i \cdot \text{out}_i$ (线性加权)
- $o = \sigma(x)$ (Sigmoid 激活)
- $\frac{\partial o}{\partial x} = o(1 - o)$
- $\frac{\partial x}{\partial w_1} = \text{out}_1$

通过逐层反向传递误差，高效计算所有参数梯度。

三、卷积神经网络 (Convolutional Neural Network, CNN)

3.1 动机

- 前馈网络处理图像需展平为向量，导致参数爆炸 (如 1000×1000 图像 $\rightarrow 10^6$ 维，全连接参数达 10^{12})
- 受 **视觉皮层分层处理机制** 启发 (Hubel & Wiesel, 1959)
- Fukushima (1980) 提出 **Neocognitron**
- LeCun (1990s) 提出 **LeNet-5**，用于手写数字识别

3.2 卷积操作 (Convolution)

- 输入图像: 5×5 灰度图
- 卷积核 (Filter/Kernel): 3×3 , 权重 w_{ij} (可学习)
- 卷积结果 (特征图, Feature Map):

$$g = w_1a + w_2b + w_3c + w_4f + w_5g + w_6h + w_7k + w_8l + w_9m$$

(以中心像素 g 为例)

卷积特性

- 局部感知 (Local Receptive Field)**: 每个输出仅依赖局部输入
- 参数共享 (Parameter Sharing)**: 同一卷积核滑动遍历整图, 大幅减少参数
- 平移不变性 (Translation Invariance)**: 无论目标在图像何处, 都能被检测

3.3 填充 (Padding) 与步长 (Stride)

- 填充 (Padding)**: 在图像边缘补0, 使输出尺寸不变
 - 例如: 5×5 图 + 3×3 核 + padding=1 $\rightarrow$ 输出仍为 5×5
- 步长 (Stride)**: 卷积核滑动步长
 - stride=1: 标准卷积
 - stride=2: 跳过像素, 下采样 (如 $5 \times 5 \rightarrow 2 \times 2$)

输出尺寸公式

设输入尺寸 W , 卷积核大小 F , 填充 P , 步长 S , 则输出尺寸:

$$W_{\text{out}} = \left\lfloor \frac{W - F + 2P}{S} + 1 \right\rfloor$$

3.4 池化 (Pooling)

降低特征图维度, 增强鲁棒性

类型	操作	作用
最大池化 (Max Pooling)	取区域最大值	保留显著特征
平均池化 (Average Pooling)	取区域均值	平滑特征
k-max 池化	取前 k 个最大值	保留多个重要响应

池化不可学习, 无参数, 但具 **下采样 + 抽象** 效果

3.5 CNN 全貌（图5.14示例）

输入: $32 \times 32 \times 3$ (RGB 图像)
→ 卷积 (6 个 $5 \times 5 \times 3$ 核, stride=1, no pad) → $28 \times 28 \times 6$
→ 池化 (2×2 max pooling) → $14 \times 14 \times 6$
→ 全连接层 → Softmax 分类

四、循环神经网络（Recurrent Neural Network, RNN）

4.1 动机与结构

- 处理 **序列数据** (文本、语音、视频)
- 引入 **记忆机制**: 当前输出依赖历史状态

基本公式（时刻 t ）：

$$h_t = \phi(Ux_t + Wh_{t-1})$$

- x_t : 当前输入
- h_{t-1} : 上一时刻隐状态 (“记忆”)
- U, W : 可学习参数
- ϕ : 激活函数 (Sigmoid / Tanh)

展开形式：等价于深度网络，每层共享参数

4.2 应用模式（图5.17）

模式	输入-输出	应用
多对多	序列 → 序列	机器翻译、摘要生成
多对一	序列 → 单值	情感分类、句子分类
一对多	单值 → 序列	图像描述生成

4.3 梯度消失与爆炸

- 由于链式求导中多次乘以 $|\tanh'| < 1$ ，长序列梯度趋近于0
- 导致 **无法学习长期依赖**

4.4 长短时记忆网络（LSTM）

引入 **门控机制** 控制信息流

核心组件

- 遗忘门 (Forget Gate) : 决定丢弃多少历史信息

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

- 输入门 (Input Gate) : 决定更新多少新信息

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

- 候选记忆:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

- 记忆单元更新:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

- 输出门 (Output Gate) :

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

- 隐状态输出:

$$h_t = o_t \odot \tanh(c_t)$$

符号说明:

- σ : Sigmoid
- $\odot$: 逐元素相乘 (Hadamard product)
- c_t : 长期记忆; h_t : 短期记忆

为何缓解梯度消失?

- 记忆更新含 加法项: $\frac{\partial c_t}{\partial c_{t-1}} = f_t + \dots$
- 若 $f_t \approx 1$, 梯度可长期传递

五、注意力机制与正则化

5.1 自注意力 (Self-Attention) — Transformer 核心

Ashish Vaswani et al., 2017

步骤 (以单词 w_3 为例) :

- 嵌入 + 位置编码 $\rightarrow w_i$
- 线性变换 (共享参数) :

$$q_i = W^q w_i, \quad k_i = W^k w_i, \quad v_i = W^v w_i$$

- 计算注意力得分:

$$\alpha_{3j} = q_3^\top k_j$$

- Softmax 归一化:

$$a_{3j} = \frac{\exp(\alpha_{3j})}{\sum_l \exp(\alpha_{3l})}$$

5. 加权求和：

$$\text{output}_3 = \sum_j a_{3j} v_j$$

特点：

- **并行计算** (vs RNN 串行)
- 捕捉任意距离依赖
- 可扩展为 **多头注意力 (Multi-head Attention)**

5.2 正则化技术 (防过拟合)

(1) Dropout

- 训练时以概率 p 随机“关闭”神经元
- 测试时使用全部神经元，权重缩放 $\times (1 - p)$
- 效果：模拟集成学习，提升泛化

(2) 批归一化 (Batch Normalization)

- 对每层输入做标准化：

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

其中 μ_B, σ_B^2 为 batch 均值/方差, γ, β 为可学习参数

- 优点：加速训练、缓解梯度消失、允许更高学习率

(3) L1 / L2 正则化

损失函数加入惩罚项：

$$\min_W [\text{Loss}(y, f(W, x)) + \lambda \cdot \Omega(W)]$$

- **L1 正则**: $\Omega(W) = \|W\|_1 = \sum |w_i| \rightarrow$ 稀疏解 (Lasso)
- **L2 正则**: $\Omega(W) = \|W\|_2^2 = \sum w_i^2 \rightarrow$ 权重衰减 (Ridge)

L0 范数 (非零参数个数) 理论上最优但 NP-Hard, 不实用

六、深度学习在 NLP 与 CV 中的应用

6.1 词向量 (Word Embedding) — Word2Vec

动机

- 传统“词袋模型 (Bag of Words)”忽略词序与语义
- One-hot 表示高维稀疏，无法衡量相似度

Word2Vec 思想

- 将词映射为 **低维稠密向量 (Distributed Representation)**
- 相似词在向量空间中距离近

模型结构 (图5.24)

- 输入: V 维 one-hot 向量 x_k
- 隐藏层: $h = W^\top x_k = w_k$ (即词向量)
- 输出: 通过另一权重矩阵 W' 映射回 V 维, 经 Softmax 得预测概率
- 训练目标: 最大化上下文预测准确率

两种训练模式

模式	输入 → 输出	说明
CBOW	上下文词 → 中心词	利用周围词预测当前词
Skip-gram	中心词 → 上下文词	利用当前词预测周围词

最终取输入层到隐藏层的权重作为词向量

6.2 图像分类与目标定位 (多任务学习)

- **共享特征提取器 (CNN)**
- **双分支输出:**
 1. **分类分支:** Softmax + 交叉熵损失
 2. **定位分支:** 回归包围盒 (x, y, w, h) + MSE 损失
- 总损失: $L = L_{cls} + L_{loc}$

实现端到端联合训练

七、本章核心思想总结

- **端到端学习:** 从原始输入直接学习到输出, 无需手工特征工程
- **层次化抽象:** 浅层捕获边缘/纹理, 深层捕获语义对象
- **数据驱动参数学习:** 通过反向传播 + 梯度下降优化
- **挑战:**
 - 模型可解释性差 (“黑箱”)
 - 依赖大量标注数据
 - 易受对抗样本攻击
- **未来方向:** 融合知识、提升可信性、减少数据依赖、增强自适应能力

“无他，但手熟尔。” —— 深度学习虽强，理解其原理方能驾驭。

✅ 本复习文档覆盖课件全部核心内容，包含所有公式、算法、模型结构及历史背景。建议结合图示（如图5.6、5.14、5.21等）加深理解。

第六章 强化学习 —— 知识点与专有名词复习文档

一、基本概念

1. 强化学习 (Reinforcement Learning, RL)

- 定义:** 智能体 (agent) 通过与环境不断交互，采用“尝试与试错”、“探索与利用”等机制，在状态中采取行动，根据终止状态获得的奖惩来改进行动策略，完成序贯决策任务。
- 特点:**
 - 基于评估:** 利用环境对当前策略进行评估并优化；
 - 交互性:** 数据在时序交互中产生；
 - 序列决策过程:** 前后决策相互关联；
 - 奖励滞后、基于采样评估。**

2. 核心要素

要素	定义
智能体 (Agent)	学习主体，依据经验做出判断并执行动作。
环境 (Environment)	智能体以外的一切；可建模为理想环境。
状态 (State)	智能体对环境的理解和编码，包含影响决策的信息。
动作 (Action)	智能体对环境施加影响的方式。
策略 (Policy)	给定状态下选择动作的依据，记为 $\pi(a s)$ 或确定性形式 $a = \pi(s)$ 。
奖励 (Reward)	序贯动作后从环境获得的收益，正表示奖励，负表示惩罚。

3. 与其他学习方式对比

学习方式	学习依据	数据来源	决策过程	学习目标
监督学习	监督信息	一次给定	单步决策（如分类）	样本 $\rightarrow$ 语义标签映射
无监督学习	数据结构假设	一次给定	无	发现数据分布模式
强化学习	环境评估	时序交互中产生	序贯决策（如博弈）	状态 $\rightarrow$ 动作映射以最大化收益

二、马尔可夫决策过程 (Markov Decision Process, MDP)

1. 定义

MDP 形式化表示为五元组：

$$\text{MDP} = (S, A, P, R, \gamma)$$

其中：

- S : 状态集合；
- A : 动作集合；
- $P(s'|s, a)$: 状态转移概率；
- $R(s, a, s')$: 奖励函数；
- $\gamma \in [0, 1]$: 折扣因子。

2. 马尔可夫性质 (Markov Property)

$$\Pr(X_{t+1} = x_{t+1} \mid X_0 = x_0, \dots, X_t = x_t) = \Pr(X_{t+1} = x_{t+1} \mid X_t = x_t)$$

3. 回报 (Return)

定义时刻 t 的回报为未来折扣奖励之和：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

4. 分段 (Episodic) vs 持续 (Continuing) 问题

- 分段问题**：轨迹有终止状态（如游戏结束）；
- 持续问题**：无终止状态（如机器人持续运行）。

三、价值函数与贝尔曼方程

1. 价值函数 (Value Function)

$$V^{\pi}(s) = \mathbb{E}[G_t \mid S_t = s]$$

表示在策略 π 下从状态 s 出发的期望回报。

2. 动作-价值函数 (Action-Value Function)

$$q^{\pi}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$

3. 贝尔曼方程 (Bellman Equation)

价值函数贝尔曼方程:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

动作-价值函数贝尔曼方程:

$$q^\pi(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a'} \pi(a'|s') q^\pi(s', a') \right]$$

四、强化学习目标

给定 MDP (S, A, P, R, γ) , 目标是找到最优策略 π^* , 使得对任意 $s \in S$, 有:

$$V^{\pi^*}(s) = \max_{\pi} V^{\pi}(s)$$

五、基于价值的强化学习 (Value-Based Methods)

1. 通用策略迭代 (Generalized Policy Iteration, GPI)

- 策略评估 (Policy Evaluation) : 计算当前策略下的价值函数;
- 策略优化 (Policy Improvement) : 基于价值函数改进策略;
- 两者交替进行直至收敛。

2. 策略优化定理 (Policy Improvement Theorem)

若对所有 s ,

$$q^\pi(s, \pi'(s)) \geq q^\pi(s, \pi(s))$$

则 $V^{\pi'}(s) \geq V^\pi(s)$, 即 π' 不劣于 π 。

构造新策略:

$$\pi'(s) = \arg \max_a q^\pi(s, a)$$

3. 算法

(1) 策略迭代 (Policy Iteration)

算法6.4 PolicyIteration

输入: MDP = (S, A, P, R, γ)

输出: 策略 π

- 随机初始化 π
- repeat
- $q \leftarrow \text{PolicyEvaluation}(\pi)$
- for each $s \in S$ do
- $\pi(s) \leftarrow \arg \max_a q(s, a)$
- end
- until π 收敛

(2) 价值迭代 (Value Iteration)

算法6.5 ValueIteration

输入: MDP = (S, A, P, R, γ)

输出: 策略 π

1. 随机初始化 V
2. repeat
3. for each $s \in S$ do
4. $V(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')]$
5. end
6. until V 收敛
7. $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')]$

4. Q-learning (无模型方法)

动作-价值函数更新规则:

$$q(s, a) \leftarrow q(s, a) + \alpha \left[R + \gamma \max_{a'} q(s', a') - q(s, a) \right]$$

算法描述 (算法6.6) :

算法6.6 QLearning

输入: MDP = (S, A, P, R, γ)

输出: 策略 π

1. 随机初始化 $q(s,a)$
2. repeat
3. $s \leftarrow$ 初始状态
4. repeat
5. $a \leftarrow \operatorname{argmax}_{a'} q(s, a')$
6. 执行 a , 观察 R 和 s'
7. $q(s,a) \leftarrow q(s,a) + \alpha [R + \gamma \max_{a'} q(s',a') - q(s,a)]$
8. $s \leftarrow s'$
9. until s 是终止状态
10. until q 收敛
11. $\pi(s) \leftarrow \operatorname{argmax}_a q(s,a)$

5. 探索与利用: ϵ -贪心策略 (ϵ -greedy)

策略定义:

$$\pi_{\epsilon}(s) = \begin{cases} \arg \max_a q(s, a), & \text{以概率 } 1 - \epsilon \\ \text{随机选择 } a \in A, & \text{以概率 } \epsilon \end{cases}$$

带 ϵ -贪心的 Q-learning (算法6.7) :

```
函数: EpsGreedy(s, q,  $\epsilon$ )
1.  $u \sim \text{Uniform}(0,1)$ 
2. if  $u \leq \epsilon$  then
3.    $a \leftarrow$  随机选择  $\in A$ 
4. else
5.    $a \leftarrow \text{argmax}_a q(s,a)$ 
6. end
```

QLearning with ϵ -greedy:

... (同 Q-learning, 第5行替换为 $a \leftarrow \text{EpsGreedy}(s, q, \epsilon)$)

六、参数化与深度强化学习

1. 参数化 Q 函数

用神经网络拟合 $q(s, a; \theta)$, 称为 **Deep Q-Network (DQN)**。

损失函数:

$$L(\theta) = \left[R + \gamma \max_{a'} q(s', a'; \theta^-) - q(s, a; \theta) \right]^2$$

其中 θ^- 是目标网络参数, 定期更新。

2. 经验回放 (Experience Replay)

- 将交互样本 (s, a, R, s') 存入回放缓冲区;
- 训练时随机采样 mini-batch, 打破样本相关性。

3. DQN 算法要点 (算法6.8 变体)

- 使用目标网络稳定训练;
- 经验回放减少方差;
- 梯度下降更新 θ 。

七、基于策略的强化学习 (Policy-Based Methods)

1. 策略参数化

设策略为 $\pi_\theta(a|s)$, 目标是最大化:

$$J(\theta) = V^{\pi_\theta}(s_0)$$

2. 策略梯度定理 (Policy Gradient Theorem)

$$\nabla_\theta J(\theta) = \sum_s \mu^\pi(s) \sum_a q^{\pi_\theta}(s, a) \nabla_\theta \pi_\theta(a|s)$$

或等价地:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s,a \sim \pi_{\theta}} [q^{\pi_{\theta}}(s,a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

其中 $\mu^{\pi}(s)$ 为状态访问频率 (假设 $\gamma = 1$)。

3. REINFORCE 算法 (蒙特卡洛策略梯度)

算法6.9 REINFORCE

输入: MDP = (S, A, P, R, γ)

输出: 策略 π_{θ}

1. 随机初始化 θ
2. *repeat*
3. 采样完整片段: $s_0, a_0, R_1, \dots, s_T$
4. *for* $t = 0$ *to* $T-1$ *do*
5. $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^k R_{k+1}$
6. $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$
7. *end*
8. *until* θ 收敛

注: 使用实际回报 G_t , 需完整 episode。

4. Actor-Critic 算法 (结合时序差分)

- **Actor**: 参数化策略 π_{θ} ;
- **Critic**: 参数化价值函数 $V_w(s)$, 用于估计 $q(s,a)$ 。

更新规则:

- Critic (价值网络):

$$w \leftarrow w + \alpha_w [R + \gamma V_w(s') - V_w(s)] \nabla_w V_w(s)$$

- Actor (策略网络):

$$\theta \leftarrow \theta + \alpha_{\theta} [R + \gamma V_w(s') - V_w(s)] \nabla_{\theta} \log \pi_{\theta}(a|s)$$

算法6.10 Actor-Critic:

输入: MDP = (S, A, P, R, γ)

输出: 策略 π_{θ}

1. 随机初始化 θ, w
2. *repeat*
3. $s \leftarrow$ 初始状态
4. *repeat*
5. $a \sim \pi_{\theta}(\cdot|s)$
6. 执行 a , 观察 R, s'
7. $\delta \leftarrow R + \gamma V_w(s') - V_w(s)$
8. $w \leftarrow w + \alpha_w \delta \nabla_w V_w(s)$
9. $\theta \leftarrow \theta + \alpha_{\theta} \delta \nabla_{\theta} \log \pi_{\theta}(a|s)$
10. $s \leftarrow s'$
11. *until* s 是终止状态
12. *until* θ 收敛

八、强化学习分类总结

维度	类型
是否依赖环境模型	Model-based / Model-free
优化对象	Value-based (优化 V 或 q) / Policy-based (直接优化 π)
价值估计方式	Dynamic Programming / Monte Carlo / Temporal Difference

九、历史与发展

- **1951**: 贝尔曼提出**贝尔曼方程**与**最优化原理**;
- **1970s**: Klopff 提出强化学习的“内在趋利”本质;
- **1980s**: Sutton & Barto 整合动态规划与时序差分, 奠定现代 RL 基础;
- **1989**: Watkins 提出 **Q-learning**;
- **2013**: DeepMind 提出 **DQN**, 开启深度强化学习时代。

十、挑战与延伸

- **奖励设计**: 稀疏、延迟、误导性;
- **样本效率低**: 需大量交互;
- **探索困境**: 易陷入局部最优;
- **训练不稳定**: 高方差、非平稳目标;
- **泛化与迁移**: 跨任务/环境能力有限。

“谋定而后动，知止而有得。”
—— 强化学习之精神内核

第七章 人工智能博弈 —— 复习文档

一、博弈论相关概念

1. 博弈行为与博弈论定义

- **博弈行为**: 多个带有相互竞争性质的主体, 为了达到各自目标和利益, 采取的对抗性行为。
- **博弈论 (Game Theory)**: 又称对策论, 研究博弈行为中最优对抗策略及其稳定局势, 协助参与者在规则范围内寻求最合理的行为方式。
- **核心思想**: “两害相权取其轻, 两利相权取其重”。

2. 历史背景

- **中国古代博弈思想**:
 - 《论语·阳货》: “不有博弈者乎? 为之, 犹贤乎已。”
 - 朱熹注: “博, 局戏; 弈, 围棋也。”

- 《围棋赋》：“略观围棋，法于用兵，怯者无功，贪者先亡。”
 - 田忌赛马：体现“以己之长，攻彼之短”的博弈思想。
- 现代博弈论奠基：
 - 1944年，冯·诺伊曼（John von Neumann）与摩根斯特恩（Oskar Morgenstern）合著《博弈论与经济行为》，标志现代博弈论初步形成。

3. 基本术语

- 参与者（Player）：博弈中的决策主体。
- 策略（Strategy）：参与者可采取的完整行动方案。
 - 纯策略（Pure Strategy）：每次选择确定策略。
 - 混合策略（Mixed Strategy）：以一定概率分布选择不同策略。
- 策略集（Strategy Set）：某参与者所有可选策略的集合。
- 局势（Outcome）：所有参与者策略组合所形成的状态。
- 收益（Payoff）：参与者在特定局势下获得的利益。
 - 混合策略下的收益为期望收益（Expected Payoff）。
- 规则（Rule）：规定行动顺序、信息获取等。

4. 博弈分类

分类维度	类型	定义
合作性	合作博弈 / 非合作博弈	是否允许参与者结盟
时间结构	静态博弈 / 动态博弈	是否同时决策或有先后顺序
信息完备性	完全信息博弈 / 不完全信息博弈	是否了解对方策略集与收益函数

5. 纳什均衡（Nash Equilibrium）

- 定义：策略组合 $\sigma^* = (\sigma_1^*, \dots, \sigma_n^*)$ ，使得对任意玩家 i ,

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i', \sigma_{-i}^*), \quad \forall \sigma_i' \in \Sigma_i$$

即任何一方单方面改变策略都不会提高自身收益。

- Nash定理：有限参与者、有限策略、实值收益函数的博弈，必存在混合策略纳什均衡。

6. 经典案例：囚徒困境

- 参与者：甲、乙
- 策略集：{沉默（合作），认罪（背叛）}
- 收益矩阵（刑期，负值表示损失）：

	乙沉默	乙认罪
甲沉默	(-0.5, -0.5)	(-10, 0)
甲认罪	(0, -10)	(-5, -5)

- 最优解：(沉默, 沉默) → 总损失最小 (-1年)

- 纳什均衡：(认罪, 认罪) → 双方理性选择导致次优结果

7. 雇主-雇员博弈（混合策略纳什均衡示例）

- 变量定义：
 - V ：雇员贡献
 - W ：工资
 - H ：雇员付出成本
 - C ：检查成本
 - F ：惩罚（没收抵押金）
- 假设： $H < W < V$, $W > C$
- 策略：
 - 雇员：偷懒 / 不偷懒
 - 雇主：检查 / 不检查
- 收益表：

	偷懒	不偷懒
检查	$(-C + F, -F)$	$(V - W - C, W - H)$
不检查	$(-W, W)$	$(V - W, W - H)$

- 设：
 - 雇主检查概率为 α
 - 雇员偷懒概率为 β
- 雇员期望收益：
 - 偷懒： $T_3 = -\alpha F + (1 - \alpha)W$
 - 不偷懒： $T_4 = W - H$
- 纳什均衡条件： $T_3 = T_4 \Rightarrow$

$$-\alpha F + (1 - \alpha)W = W - H \Rightarrow \alpha = \frac{H}{W + F}$$

- 雇主期望收益：
 - 检查： $T_1 = \beta(-C + F) + (1 - \beta)(V - W - C)$
 - 不检查： $T_2 = -\beta W + (1 - \beta)(V - W)$
- 均衡条件： $T_1 = T_2 \Rightarrow$

$$\beta = \frac{C}{W + F}$$

- 雇主最大收益：对 W 求导得最优工资：

$$W^* = \sqrt{CV} - F, \quad T_{\max} = V - 2\sqrt{CV} + F$$

8. 策梅洛定理 (Zermelo's Theorem)

- 适用范围：双人、有限步、完全信息、零和、动态博弈
- 结论：必存在以下之一：
 - 先手必胜策略
 - 后手必胜策略
 - 双方保平策略
- 证明方法：数学归纳法（基于博弈树深度 N ）
- 注意：不适用于三人及以上博弈。

二、博弈策略求解

1. 最优反应策略 (Best Response)

- 给定其他玩家策略 σ_{-i} ，玩家 i 的最优反应 σ_i^* 满足：

$$u_i(\sigma_i^*, \sigma_{-i}) \geq u_i(\sigma'_i, \sigma_{-i}), \quad \forall \sigma'_i \in \Sigma_i$$

- 若所有玩家策略互为最优反应，则构成纳什均衡。

2. 虚拟遗憾最小化算法 (Counterfactual Regret Minimization, CFR)

(1) 基本遗憾最小化 (Regret Matching)

- 累加遗憾值（玩家 i 在 T 轮中对策略 a ）：

$$\text{Regret}_i^T(a) = \sum_{t=1}^T [u_i(a, \sigma_{-i}^t) - u_i(\sigma_i^t)]$$

- 有效遗憾值：

$$\text{Regret}_i^{T,+}(a) = \max(\text{Regret}_i^T(a), 0)$$

- 第 $T+1$ 轮选择策略 a 的概率：

$$P^{T+1}(a) = \begin{cases} \frac{\text{Regret}_i^{T,+}(a)}{\sum_{a' \in A} \text{Regret}_i^{T,+}(a')} & \text{if } \sum \text{Regret}^+ > 0 \\ \frac{1}{|A|} & \text{otherwise} \end{cases}$$

(2) 虚拟遗憾值 (Counterfactual Regret)

- 博弈树模型：
 - 节点 = 信息集 I
 - 边 = 行动 $a \in A(I)$
- 行动序列路径 h ：从根到当前节点的路径
- 到达信息集 I 的概率（忽略玩家 i 策略）：

$$\pi_{-i}^{\sigma}(h) = \prod_{j \neq i} \sigma_j(a_j)$$

- 从 h 到终局 z 的概率: $\pi^{\sigma}(h, z)$
- 虚拟价值 (Counterfactual Value) :

$$v_i^{\sigma}(h) = \sum_{z \in Z} \pi_{-i}^{\sigma}(h) \cdot \pi^{\sigma}(h, z) \cdot u_i(z)$$

- 单步遗憾值 (在路径 h 下选择行动 a) :

$$r_i(h, a) = v_i^{\sigma|_{I \rightarrow a}}(h) - v_i^{\sigma}(h)$$

- 信息集 I 的总遗憾值:

$$r_i(I, a) = \sum_{h \in I} r_i(h, a)$$

- 累加虚拟遗憾值 (T 轮后) :

$$\text{Regret}_i^T(I, a) = \sum_{t=1}^T r_i^t(I, a)$$

- 更新策略 (遗憾匹配) :

$$\sigma^{T+1}(I, a) = \begin{cases} \frac{\text{Regret}_i^{T,+}(I, a)}{\sum_{a' \in A(I)} \text{Regret}_i^{T,+}(I, a')} & \text{if } \sum \text{Regret}^+ > 0 \\ \frac{1}{|A(I)|} & \text{otherwise} \end{cases}$$

(3) CFR 算法步骤

1. 初始化所有遗憾值和累计策略为 0。
2. 使用随机策略进行一轮博弈。
3. 对博弈树中每个信息集, 计算虚拟价值与遗憾值。
4. 更新每个信息集的累加遗憾值。
5. 根据遗憾匹配更新策略。
6. 重复 2-5 多轮。
7. 最终策略为各轮策略的平均 (或最后几轮)。

3. 库恩扑克 (Kuhn Poker) 示例

- 规则:
 - 两人, 三张牌 $\{1, 2, 3\}$
 - 每人发一张, 轮流过牌 (P) 或加注 (B)
 - 收益见教材表 7.8
- 信息集: 共 12 个, 如 $\{1PB\}$ 表示持牌 1, 先过牌, 对手加注, 再轮到自己
- CFR 计算示例 (信息集 $\{1PB\}$) :
 - 当前策略下虚拟价值: $v^{\sigma}(h) = -0.75$
 - 若选择“过牌”: $v^{\sigma|_P}(h) = -0.5$
 - 遗憾值: $r(P) = -0.5 - (-0.75) = 0.25$

- “加注”遗憾值: $r(B) = -0.25$
- **最终策略** (表 7.9) 给出每个信息集选择 B 或 P 的概率。

4. 安全子博弈 (Safe Subgame)

- **完全信息博弈**: 子博弈可独立求解 (与全局无关)
- **非完全信息博弈**: 子博弈与全局相关, 不能孤立处理
- **安全子博弈定义**: 在子博弈中求解的结果**不劣于**全局近似解
- **抛硬币游戏示例**:
 - 玩家 A 抛硬币 (正/反), 可选择卖 (收益 ± 0.5) 或让 B 猜 (猜错 A 得 +1, 猜对 A 得 -1)
 - 若 B 仅考虑猜硬币子博弈, 会以 0.5 概率猜正/反
 - 但**安全策略**需考虑 A 卖硬币的收益, 最终 B 应以 0.25 概率猜正面 (使 A 期望收益为 0)

三、博弈规则设计

1. 研究问题

如何设计规则, 使个体理性选择导向**整体利益最大化**?

2. 双边匹配问题 (Two-sided Matching)

- **典型场景**: 求职、高校录取、器官分配
- **稳定婚姻问题 (Stable Marriage Problem)**
 - 输入: n 男 $M = \{m_1, \dots, m_n\}$, n 女 $F = \{f_1, \dots, f_n\}$, 每人有偏好列表
 - **稳定匹配**: 不存在一对 (m, f) 彼此更喜欢对方而非当前伴侣 (即无“阻塞对”)

Gale-Shapley 算法 (G-S Algorithm)

1. 所有单身男性向**最偏好**且未拒绝过的女性求婚。
2. 每位女性从求婚者中选择**最偏好者**, 暂时配对 (可抛弃前任)。
3. 重复直至无人单身。

- **性质**:
 - 总能找到**稳定匹配**
 - **男性最优** (在所有稳定匹配中, 男性获得最佳可能伴侣)
 - **女性最劣**

3. 单边匹配问题 (One-sided Matching)

- **场景**: 床位分配、室友匹配、物品交换
- **特点**: 标的物不可分, 初始有占有者

最大交易圈算法 (Top-Trading Cycle, TTC)

1. 每人指向**最偏好**的物品。
2. 每物品指向其当前占有者。
3. 图中必存在**有向环 (交易圈)**。

- 4. 圈内成员交换物品，退出市场。
 - 5. 重复直至无剩余。
 - **示例：**四人 A,B,C,D 分配床位 1-4，初始分配 (4,3,2,1)
 - 第一轮：A→1→D→4→A ⇒ 交易圈 (A,D)
 - A 得 1，D 得 4
 - 剩余 B,C 与床位 2,3 ⇒ C 是 2 的占有者，保留；B 得 3
-

四、非完全信息博弈

1. 特点

- 参与者**不掌握全部信息**（如对手手牌、私有状态）
- 需在**信息集 (Information Set)** 上制定策略（同一信息集内无法区分历史）

2. 典型应用：德州扑克 AI

- **Libratus**（双人无限注德州扑克 AI）：
 - 使用 **CFR** 进行自我博弈训练
 - 实战中采用**安全子博弈求解**缩小搜索空间
 - 结合**强化学习**反馈优化策略
 - **Pluribus**（六人德州扑克 AI）：
 - 使用 **蒙特卡洛虚拟遗憾最小化 (MCCFR)**
 - 随机采样部分行动，降低计算量
 - 采用**有限前瞻搜索 (Depth-limited Search)**
 - 截断搜索，自适应应对对手策略变化
-

五、延伸与总结

1. 人工智能与博弈论融合

- 推动 AI 从**感知智能** → **决策智能**
- 从“求最优解” → “求均衡解”
- 核心方向：
 - 算法博弈论 & 计算经济学
 - 多智能体强化学习
 - 博弈驱动的机器学习理论

2. 算法博弈 vs 古代占卜

- **古代：**用骰子、龟甲规避命运不确定性
- **现代：**用**算法博弈**以数据驱动方式应对随机性与策略对抗

3. 本章核心公式汇总

概念	公式
纳什均衡	$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i', \sigma_{-i}^*)$
雇主检查概率	$\alpha = \frac{H}{W + F}$
雇员偷懒概率	$\beta = \frac{C}{W + F}$
最优工资	$W^* = \sqrt{CV} - F$
虚拟价值	$v_i^\sigma(h) = \sum_z \pi_{-i}^\sigma(h) \pi^\sigma(h, z) u_i(z)$
虚拟遗憾	$r_i(h, a) = v_i^{\sigma _{I \rightarrow a}}(h) - v_i^\sigma(h)$
遗憾匹配概率	$P(a) = \frac{\text{Regret}^+(a)}{\sum \text{Regret}^+}$

注：本复习文档严格依据课件内容整理，覆盖全部专有名词、算法、公式与案例，适用于系统性复习与考试准备。

《人工智能引论》第八章：人工智能伦理与安全 —— 复习文档

一、人工智能伦理

1.1 伦理概念起源

- 词源：
 - 西方：“伦理”源于希腊语 *ethos*（性格）和拉丁语 *mores*（风俗），指人与人之间言行的道德准则。
 - 中文：“伦”表示人际关系，“理”表示规则秩序。
 - 《礼记·乐记》：“乐者，通伦理者也。”
 - 许慎《说文解字》：“伦，辈也”，强调至少两人以上的关系。
 - 郑玄注：“伦，犹类也，理之分也”，强调对“类”的分辨。
- 伦理学定义：对道德、道德问题及道德判断所作的哲学思考，是哲学的一个分支。

1.2 科技的双重属性

- 技术先行路径（Proactionary Approach）：以发展技术为优先，强调工具理性（效率/效用最大化），易导致科技异化。
- 科技伦理兴起：在科学的社会建构思潮下，要求兼顾技术属性与社会属性。
- 三元空间结构 CPH（Cyber-Physical-Human）：
 - 传统二元结构：物理世界 + 人类社会；
 - 新型三元结构：信息空间（Cyber）+ 物理世界（Physics）+ 人类社会（Human）。

- 人工智能在此结构中需嵌入人类价值观、道德观和法律法规。

1.3 科林格里奇困境 (Collingridge's Dilemma)

- **提出者**：英国技术哲学家 David Collingridge (1980, 《The Social Control of Technology》)
- **核心观点**：
 - 技术早期难以预测其社会后果；
 - 待发现问题时，技术已深度嵌入社会经济结构，难以控制。
- **启示**：需在技术早期就引入伦理治理机制。

1.4 普遍智能 (General Intellect)

- **马克思观点**：机器体系是“人手创造出来的人脑器官”，既是异己力量，又受控于全社会共享的“普遍智能”。

二、人工智能模型安全

2.1 对抗攻击 (Adversarial Attack)

定义

在输入样本中添加人类无法察觉的微小扰动，使模型以高置信度输出错误结果。

示例

- 原图“熊猫”识别为 57.7%；
- 添加微小噪声后，被识别为“长臂猿” 99.3%。

对抗样本性质

- **存在性**：对抗样本普遍存在；
- **可迁移性 (Transferability)**：在一个模型上生成的对抗样本，对其他模型也有效。

对抗样本生成优化目标

给定图像 $\mathbf{x} \in [0, 1]^m$ ，真实标签 y ，目标标签 $y' \neq y$ ，寻找最小扰动 δ ：

$$\begin{aligned} \min_{\delta} \quad & \|\delta\|_2 \\ \text{s.t.} \quad & f(\mathbf{x} + \delta) = y' \\ & \mathbf{x} + \delta \in [0, 1]^m \end{aligned}$$

具体算法

(1) L-BFGS 方法

将约束优化转化为无约束形式：

$$\min_{\delta} \quad c\|\delta\| + \mathcal{L}_{\text{CE}}(\mathbf{x} + \delta, y') \quad \text{s.t.} \quad \mathbf{x} + \delta \in [0, 1]^m$$

其中：

- $\mathcal{L}_{\text{CE}}$ 为交叉熵损失；
- c 通过线搜索确定。

(2) 快速梯度符号法 (FGSM, Fast Gradient Sign Method) [Goodfellow et al., 2014]

$$\mathbf{x}' = \mathbf{x} + \delta = \mathbf{x} + \eta \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}), y))$$

- η : 步长 (学习率) ;
- $\text{sign}(\cdot)$: 符号函数;
- 沿损失函数梯度方向最大化损失。

(3) 投影梯度下降 (PGD, Projected Gradient Descent) [Madry et al., 2017]

迭代形式:

$$\mathbf{x}_{k+1} = \Pi_{\mathcal{B}(\mathbf{x}, \epsilon)} (\mathbf{x}_k + \alpha \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}_k), y)))$$

- Π : 投影到以 $\mathbf{x}$ 为中心、半径为 ϵ 的范数球 (如 L_{∞}) ;
- $\alpha < \eta$, 多步迭代, 比 FGSM 更强;
- 是当前对抗鲁棒性评估的标准基线。

攻击类型

类型	是否需模型梯度	描述
白盒攻击	是	攻击者完全知道模型结构与参数
黑盒攻击	否	仅能访问输入/输出
- 基于迁移性	否	利用替代模型生成可迁移对抗样本
- 基于查询	否	通过有限差分等方法估计梯度

2.2 数据投毒攻击 (Data Poisoning Attack)

定义

在训练数据收集阶段注入恶意样本，干扰模型训练，降低其性能（整体或特定类别）。

分类

- 标签反转投毒 (Label-flipping) : 将样本标签改为错误类别（如垃圾邮件标为非垃圾）。
- 干净标签投毒 (Clean-label) : 标签正确，但样本被修改（如加不可见噪声），更隐蔽。

形式化描述

- 干净训练集: $\mathcal{D}_{\text{train}}^{\text{clean}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$
- 投毒样本构造:

$$\mathbf{x}^* = f_{\text{poison}}(\mathbf{x}), \quad y^* = l_{\text{poison}}(y)$$

- 最终训练集: $\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{train}}^{\text{clean}} \cup \mathcal{D}_{\text{poison}}$

防御方法

1. 基于投毒样本检测

- 利用特征分布差异（如高层特征接近目标类别）；
- 使用 **k近邻** 或 **Influence Function** 检测异常样本。

2. 模型鲁棒性增强

- 使用 **SGD** 优化器具有一定天然鲁棒性；
 - **DP-SGD (差分隐私 SGD)**：限制梯度范数 + 添加噪声。
-

2.3 后门攻击 (Backdoor Attack)

定义

数据投毒的特殊形式：模型在正常输入下表现正常，但当输入包含**触发器 (trigger)** 时，输出被操控为目标类别。

示例

- 交通灯识别任务中，在 STOP 标志上加黄色矩形 → 模型输出 “80km/h”。

形式化构造

- 触发器： t
- 投毒函数： $\mathbf{x}^* = f_{\text{poison}}(\mathbf{x}, t)$
- 标签设为目标类别 y_b
- 训练集： $\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{clean}} \cup \{(\mathbf{x}^*, y_b)\}$

防御方法

1. 针对输入触发器

- 预处理净化输入；
- 如文本后门中，使用 **语言困惑度 (Perplexity)** 检测并移除触发词 (ONION 方法 [Qi et al., 2021])。

2. 针对模型内嵌后门

- 使用干净数据 **微调/重训练** 模型；
 - 利用 **灾难性遗忘 (Catastrophic Forgetting)** 消除后门 ([Liu et al., 2017])。
-

2.4 隐私保护

隐私定义

- 1890年《哈佛法律评论》：“不受打扰的权利”。
- 匿名化不足：可通过数据库交叉比对去匿名（如马萨诸塞州州长健康记录泄露事件）。

隐私保护技术

技术	描述
差分隐私 (Differential Privacy, DP)	通过加噪提供可量化隐私保障
同态加密 (Homomorphic Encryption)	允许在密文上直接计算
安全多方计算 (Secure Multi-Party Computation)	多方协作计算而不泄露各自数据

差分隐私 (DP) 定义

一个随机算法 M 满足 (ϵ, δ) -DP, 若对任意两个**相邻数据集** D 和 D' (仅相差一条记录), 以及任意输出子集 O , 有:

$$\Pr[M(D) \in O] \leq e^\epsilon \Pr[M(D') \in O] + \delta$$

- ϵ : 隐私预算 (越小隐私越强) ;
- δ : 失败概率 (通常 $\delta \approx 0$) 。

实现方式

- 在 **目标函数**、**梯度** 或 **输出** 中添加噪声 (如 Laplace、Gaussian 噪声) ;
- 目标: 保留统计特性, 去除个体特征。

三、人工智能可解释性 (Explainable AI / Interpretable AI)

3.1 定义

- 可解释 AI: 能清晰说明其**决策过程**与**结果原因**, 建立人类信任。
- 与“黑盒模型”相对 (如深度神经网络通常缺乏透明性) 。

3.2 研究方向

(1) 神经网络中层特征的语义可解释性

- 研究中层神经元是否表达明确语义概念;
- 尝试将特征解构为基本语义单元;
- 自动学习语义清晰的中层表示。

注: 卷积网络中, 单个高层神经元与一组神经元线性组合的语义表达能力相近。

(2) 决策逻辑的可解释性

- 提取模型的因果逻辑、线性加权逻辑等;
- 例如: 哪些特征导致分类为“猫”?

(3) 基于可解释性的中层对端模型学习

- 利用中层语义特征进行**弱监督学习**；
- 可直接迁移/修改预训练特征；
- 在语义层面诊断模型错误；
- 泛化能力强，样本需求少。

3.3 可解释性 vs 符号主义 vs 连接主义

- **符号主义 AI**：基于逻辑推理（如苏格拉底三段论），天然可解释；
- **深度学习（连接主义）**：基于概率输出，缺乏显式推理链；
- 因此，可解释性研究对深度学习尤为重要。

四、延伸思想与本章小结

4.1 《人有人的用处》—— Norbert Wiener（1950）

- 强调人在技术世界中的尊严与主体性；
- 机器是人创造的工具，人应不断用知识强化机器；
- 反对“人机相斗”或“AI奴役人类”的幻想；
- 主张**人机协同、共同进化**。

4.2 本章核心观点总结

- **技术无罪，人心有责**（亚里士多德：一切实践以“善”为目标）；
- AI具有**技术属性与社会属性**，必须嵌入伦理与法律；
- **科林格里奇困境**警示：伦理治理需前置；
- **模型安全**三大威胁：对抗攻击、数据投毒、后门攻击；
- **防御策略**需覆盖训练与测试阶段；
- **隐私保护**需超越匿名化，采用 DP 等严格机制；
- **可解释性**是建立人机信任的关键桥梁；
- **人类始终是 AI 的总开关与协调者**。

结语：加强人工智能伦理建设，提供坚实安全技术保障，将为人类社会的“多样性”打开新的窗户——因为“人有人的用处”。

第九章 人工智能架构与系统 —— 复习文档

基于《人工智能引论》教材课件（吴飞，浙江大学）

一、人工智能基础软硬件框架

1. 智能计算支持框架的分层结构

开发系统 (Development System)

- **人工智能芯片**：提供物理算力支撑。
- **基础软件与计算系统**：包括操作系统中的文件系统、内核调度等。
- **训练框架**：执行 AI 所需的计算任务，如：
 - 矩阵乘法
 - 数据通信
 - 计算图执行
- **开发环境**：封装底层能力，提供用户友好的编程接口（如 Python）。

运行系统 (Runtime System)

- **芯片指令集标准**：定义硬件可执行指令，确保软硬件兼容。
- **AI 编译框架**：将高级模型描述（如 ONNX、TensorFlow Graph）编译为底层硬件指令。
- **关键算法与模型**：如深度学习模型、机器学习算法。
- **AI 垂直领域开放创新平台**：面向特定行业（医疗、工业、农业、城市治理等）提供解决方案和工具链。

二、人工智能芯片

1. GPU (Graphics Processing Unit)

- 最初用于图形渲染，因其**大规模并行计算架构**被广泛用于 AI。
- 特点：
 - 成百上千个计算核心
 - 高效执行矩阵/向量运算
- 历史事件：
 - **2006年**：Kumar Chellapilla 使用 GeForce 7800 实现 CNN，速度比 CPU 快 4 倍。
 - **2012年**：AlexNet (6000 万参数) 在 4 颗 GTX 580 上完成训练，总计算量达 **262 千万亿次浮点运算 (262 PFLOPs)**。
- 架构命名惯例（英伟达）：
 - Curie (2004)、Tesla (2006)、Fermi (2010)、Kepler (2012)、Maxwell (2014)、Pascal (2016)、Volta (2017)、Turing (2018)、Ampere (2020)、Hopper (2022)、Ada (2023)

2. TPU (Tensor Processing Unit)

- 谷歌自研专用 AI 加速器。
- 针对**张量运算**（尤其是矩阵乘加）优化。
- 特点：

- 高内存带宽
- 低功耗、低延迟
- 支持多种深度学习框架
- 应用场景：语音识别、图像处理、NLP 等。

3. XPU（可扩展处理器）

- 集成多种处理器单元（CPU + GPU + DSP 等）。
- 目标：**异构协同计算**，按任务类型动态分配计算资源。

4. 类脑芯片（Neuromorphic Chip）

- 受生物神经元启发，采用**事件驱动**和**脉冲神经网络（SNN）**架构。
- 特点：
 - 超低功耗
 - 高并行性
 - 适用于模式识别、感知、自适应学习

5. 特定领域芯片（Domain-Specific Chip）

- 针对某一类 AI 任务（如 NLP、CV）定制的 ASIC。
- 代表：华为昇腾（Ascend）、寒武纪 MLU 等。

三、人工智能编程框架

主流深度学习框架：

框架	主导方	特点
TensorFlow	Google	静态图、生产部署强、生态完善
PyTorch	Meta (Facebook)	动态图、调试友好、学术研究首选
PaddlePaddle	百度	工业级、端到端套件、中文生态好
MindSpore	华为	端云协同、自动并行、与昇腾芯片深度协同

四、分布式神经网络训练算法与系统

1. 并行策略

(1) 数据并行（Data Parallelism）

- **原理**：每个计算节点持有**完整模型副本**，但只处理**部分数据**。
- **前向传播**：各节点独立计算。
- **反向传播**：各节点计算本地梯度后，需**同步梯度或模型参数**。
- **通信瓶颈**：
 - 高频梯度同步 → 网络带宽压力大

- Batch size 问题:
 - 若单节点 batch 固定 → 总 batch 随节点数增大 → 影响收敛
 - 若总 batch 固定 → 单节点 batch 减小 → 计算资源利用率下降

图9.2: 数据并行示意图 (略)

(2) 张量并行 (Tensor Parallelism / Model Parallelism)

- 适用场景: 模型过大, 无法存入单节点内存。
- 原理: 将单层神经网络的权重矩阵切分到多个设备上。
 - 例如: 将全连接层 $W \in \mathbb{R}^{m \times n}$ 按列切分为 $W_1, W_2, \dots$
- 通信需求: 每层前/反向传播需交换中间激活值或梯度。
- 限制: 仅在节点内多加速器 (如同一服务器内多 GPU) 间高效, 跨节点通信延迟高。

图9.3: 张量并行示意图 (略)

(3) 流水线并行 (Pipeline Parallelism)

- 原理: 将模型按层划分为多个阶段 (stage), 每个 stage 分配给一个设备。
 - 例如: 5 层网络 → 分为 3 个 stage: [L1], [L2-L3], [L4-L5]
- 执行方式: 类似 CPU 流水线, 不同 micro-batch 在不同 stage 并行处理。
- 要求:
 - 各 stage 计算负载均衡
 - 相邻 stage 间需传递激活值/梯度 → 需低延迟、高带宽网络
- 挑战: 存在“气泡” (bubble) 时间, 降低设备利用率。

图9.4: 流水线并行示意图 (略)

2. 通信与同步机制

(1) 参数服务器 (Parameter Server, PS)

- 架构:
 - 中央化参数存储 (可分布式)
 - 计算节点上传梯度 → PS 更新参数 → 广播新参数
- 优点: 计算节点间无直接通信, 同步开销低。
- 缺点: PS 节点成为通信瓶颈, 尤其当计算节点 $\gg$ PS 节点时。

图9.5: 参数服务器拓扑 (略)

(2) Ring AllReduce

- 原理:
 - 将 N 个计算节点组成环形拓扑
 - 每次 AllReduce 需 $2(N - 1)$ 次点对点通信
 - 每节点局部归约 + 传递
- 优点:

- 无中心节点
- 网络负载均衡
- 高集群带宽利用率
- **缺点**：节点数增加 → 通信延迟线性增长

图9.6: Ring AllReduce (略)

3. 资源管理与调度

- **工具**：Kubernetes、Apache Mesos
- **目标**：设备分配、任务调度、负载均衡
- **GPU 多租户挑战**：
 - 资源利用率低（许多任务未满载）
 - “组调度”需求（任务需同时获得多个 GPU）→ 排队等待时间长
- **解决方案** (Xiao et al., 2020) :
 - **细粒度控制**：分离 GPU 内存与计算单元调度
 - **协同调度器**：深度学习框架 + 集群调度器联合优化

4. 容错与恢复

- 分布式训练中需处理：
 - 节点故障
 - 网络中断
- 机制：
 - Checkpointing（定期保存模型状态）
 - 自动重试与任务迁移

五、性能瓶颈与优化方向

1. “剪刀型”性能发展曲线

- **算力 (FLOPs)**：每 2 年增长 **3.1 倍** → 20 年增长 **9 万倍**
- **DRAM 带宽 & 网络带宽**：每 2 年仅增长 **1.4 倍** → 20 年增长 **30 倍**
- **后果**：**通信成为瓶颈**，尤其在大规模分布式训练中

图9.7: 硬件性能增长对比 (FLOPs、访存、通信、晶体管数)

2. 网内计算 (In-Network Computing)

- **动机**：在数据传输过程中**直接进行梯度规约**，减少通信量
- **代表工作**：SwitchML (Sapio et al., 2021)
 - 利用**可编程智能交换机**（如 P4 交换机）
 - 计算节点上传梯度 → 交换机执行 AllReduce → 返回结果

- **优势：**
 - 结合 PS（集中）与 Ring（去中心）优点
 - 通信复杂度降至 $O(1)$ per node
- **局限：**交换机计算资源有限，需软硬协同演进

六、计算机体系结构背景知识（延伸阅读）

1. 摩尔定律（Moore's Law）

- 表述：集成电路上晶体管数量每 **18-24 个月翻倍**，性能相应提升。
- 现状：增速放缓，但尚未完全失效。

2. 登纳德缩放定律（Dennard Scaling）

- 表述：晶体管缩小 → 功率密度不变 → 能效提升。
- **终结时间：**约 **2007 年开始放缓**，**2012 年基本失效**
- 后果：无法再靠工艺缩放自动提升能效 → **需架构级创新**

3. 阿姆达尔定律（Amdahl's Law）

- 公式：

$$S = \frac{1}{(1 - p) + \frac{p}{s}}$$

其中：

- S ：系统加速比
 - p ：可并行部分占比
 - s ：该部分的加速倍数
- 启示：**系统整体加速受限于串行部分**

七、大模型带来的挑战

- **模型规模爆炸：**
 - AlexNet（2012）：6000 万参数
 - GPT-3（2020）：**1750 亿参数**
- **算力需求增长：**每 **3.4 个月翻倍**
- **四大“墙”限制冯·诺依曼架构：**
 1. **计算墙：**算力不足
 2. **内存墙：**带宽/容量不足
 3. **通信墙：**节点间同步慢
 4. **能耗墙：**功耗过高

解决路径：**存算一体、近存计算、光计算、类脑架构**等新型体系结构

八、本章核心思想总结

- “数据是燃料、模型是引擎、算力是加速器”
- AI 系统 = 算法 + 芯片 + 编译器 + 框架 + 应用的深度融合
- 未来趋势：
 - 软硬协同设计
 - 领域专用架构 (DSA)
 - 开放指令集 (如 RISC-V)
 - 敏捷芯片开发

引用：“干预算子 (do-operator) 改变世界本身”——AI 系统正是当今改变世界的基础设施。

✅ **复习建议**：重点掌握三种并行策略、通信机制对比、芯片类型特点、以及性能瓶颈分析。公式虽少，但阿姆达尔定律和 FLOPs/带宽增长趋势需理解其工程含义。

《人工智能引论》第十章：人工智能应用 —— 复习文档

一、语言基础模型（大模型）

1. 自然语言处理范式演变

- 早期方法**：基于语言学规则（如乔姆斯基文法、产生式规则）
- 机器学习方法**：任务特定模型训练（如情感分析、命名实体识别等）
- 预训练 + 微调范式**：
 - 利用大规模语料进行预训练
 - 在特定任务上微调
- 预训练 + 提示学习 (Prompt Learning) + 指令微调 (Instruction Tuning)**
 - 将下游任务转化为自然语言指令形式
 - 提高零样本 (zero-shot) 和少样本 (few-shot) 泛化能力

2. 语言模型建模原理

给定长度为 T 的单词序列 $\mathbf{x} = (w_1, w_2, \dots, w_T)$ ，传统语言模型通过**下一个词预测** (Next Word Prediction, NWP) 建模联合概率分布：

$$p(w_1, w_2, \dots, w_T) = \prod_{i=1}^T p(w_i | w_1, w_2, \dots, w_{i-1})$$

- N-gram 模型**：采用马尔可夫假设，仅考虑有限上下文
- 神经网络语言模型 (NNLM)**：使用 RNN 捕捉长距离依赖
- Transformer 架构**：取代 RNN，支持并行计算与长程记忆

Transformer 内嵌表示

每个单词 w_i 被映射为内嵌向量 (embedding)，并加入位置编码 (Positional Encoding) 以保留顺序信息。

3. 三类语言大模型架构

(1) 编码模型 (Encoder-only)

- 代表模型：BERT、RoBERTa
- 预训练目标：**掩码语言建模** (Masked Language Modeling, MLM)

输入部分词被替换为 [MASK]，模型预测被掩码的词：

$$\mathcal{L}_{\text{MLM}}(\theta; \mathbf{x}) = \sum_{t=1}^T \log P(w_t \mid \mathbf{x}_{\setminus t}; \theta)$$

其中 $\mathbf{x}_{\setminus t}$ 表示第 t 个词被掩码后的输入。

缺点：

- 预训练与微调阶段不一致（微调时无 [MASK]）
- 不适合生成任务（非自回归）

(2) 解码模型 (Decoder-only)

- 代表模型：GPT-2、GPT-3
- 自回归建模：仅使用左侧上下文预测下一个词

$$\mathcal{L}_{\text{LM}}(\theta; \mathbf{x}) = \sum_{t=1}^T \log P(w_t \mid \mathbf{x}_{<t}; \theta)$$

其中 $\mathbf{x}_{<t} = (w_1, \dots, w_{t-1})$

优点：适合文本生成（对话、摘要等）

缺点：无法利用双向上下文，理解能力受限

(3) 编码-解码模型 (Encoder-Decoder)

- 代表模型：BART、T5
- 输入被“破坏”（如掩码、删除、打乱），模型重建原始序列

训练目标：

$$\mathcal{L}_{\text{LM}}(\theta; \mathbf{x}) = \sum_{t=u}^v \log P(w_t \mid \mathbf{x}_{\setminus [u,v]}; \theta)$$

其中 $\mathbf{x}_{\setminus [u,v]}$ 表示被破坏的输入， $[u, v]$ 为需重建区间。

优势：兼顾理解与生成能力

4. Transformer 机制详解 (以 GPT-3 为例)

- 每层包含：
 - 多头自注意力机制 (Multi-Head Self-Attention)
 - 前馈神经网络 (Feed-Forward Network)
- 参数占比：
 - 注意力模块 $\approx 30\%$
 - 前馈网络 $\approx 60\%$

自注意力计算流程

对每个词 w_i , 计算其 Query、Key、Value:

$$\begin{aligned} \mathbf{q}_i &= \mathbf{W}_Q \mathbf{w}_i \\ \mathbf{k}_j &= \mathbf{W}_K \mathbf{w}_j \\ \mathbf{v}_j &= \mathbf{W}_V \mathbf{w}_j \end{aligned}$$

计算注意力得分:

$$\text{score}_{ij} = \mathbf{q}_i^\top \mathbf{k}_j$$

归一化得注意力权重:

$$\alpha_{ij} = \text{softmax}_j(\text{score}_{ij}) = \frac{\exp(\text{score}_{ij})}{\sum_k \exp(\text{score}_{ik})}$$

加权求和得输出:

$$\text{output}_i = \sum_{j=1}^L \alpha_{ij} \mathbf{v}_j$$

5. 指令微调 (Instruction Tuning) vs 提示学习 (Prompt Learning)

特性	提示学习 (GPT-3)	指令微调
是否更新模型参数	否	是
依赖人工模板	是	否 (或弱依赖)
零样本性能	一般	显著提升
对齐人类意图	弱	强

指令微调将多种 NLP 任务统一为“指令→输出”格式，提升泛化能力。

6. 人在环路反馈 (RLHF: Reinforcement Learning with Human Feedback)

用于对齐模型输出与人类价值观（安全、诚实、无偏见等）。

两阶段流程：

(1) 训练奖励模型 (Reward Model, RM)

- 用 SFT (Supervised Fine-Tuned) 模型生成多个回答
- 人工对回答排序（偏好标注）
- 训练 RM 学习人类偏好

损失函数 (pairwise ranking loss)：

$$\mathcal{L}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim D} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

其中：

- y_w ：更优回答， y_l ：较差回答
- σ ：Sigmoid 函数
- $r_\theta(x, y)$ ：RM 对回答 y 的打分

使用排序而非绝对打分，因排序更稳定、标注一致性更高。

(2) PPO 优化策略模型

用 PPO (Proximal Policy Optimization) 算法，基于 RM 奖励优化 SFT 模型：

$$\mathcal{L}^{\text{RL}}(\phi) = \mathbb{E}_{(x, y) \sim D_{\pi^\phi}} \left[r_\theta(x, y) - \beta \log \frac{\pi^\phi(y | x)}{\pi^{\text{SFT}}(y | x)} \right] + \gamma \mathbb{E}_{x \sim D_{\text{pretrain}}} [\log \pi^\phi(x)]$$

其中：

- 第一项：最大化 RM 奖励
- 第二项：KL 散度约束，防止偏离 SFT 过远
- 第三项（可选）：保留预训练语言建模能力 (PPO-ptx)

二、自然语言中的机器翻译

1. 发展阶段

- 基于文法 (Grammar-based)：依赖语言学规则，解析 → 生成
- 基于统计 (SMT)：利用双语平行语料，学习词对齐与翻译概率
- 基于深度学习 (NMT)：端到端神经网络模型

2. 编码器-解码器框架 (RNN-based)

- 编码器：将源句 $\mathbf{x} = (x_1, \dots, x_T)$ 编码为上下文向量 $\mathbf{h}_T$
- 解码器：以 $\mathbf{h}_T$ 为初始状态，自回归生成目标句 $\mathbf{y} = (y_1, \dots, y_{T'})$

训练目标：最小化交叉熵损失

$$\mathcal{L} = - \sum_{t=1}^{T'} \log P(y_t | y_{<t}, \mathbf{x}; \theta)$$

后续改进引入 **注意力机制** (Bahdanau et al., 2015) , 解决长句信息瓶颈问题。

三、图像分类与视觉对象定位

1. 卷积神经网络 (CNN) 结构

- 卷积层**: 提取局部特征 (使用卷积核滑动)
- 池化层**: 降维, 保留显著特征 (最大池化 / 平均池化)
- 全连接层**: 用于分类或回归

2. 多任务学习示例

- 分类任务**: Softmax 输出类别概率

$$P(c | \mathbf{I}) = \frac{\exp(z_c)}{\sum_k \exp(z_k)}$$

- 定位任务**: 输出边界框坐标 (x, y, w, h)

3. Faster R-CNN 流程

- 区域候选网络 (RPN)**: 生成候选框
- RoI Pooling**: 将不同尺寸候选区域映射为固定大小特征
- 分类 + 回归**: 对每个 RoI 进行类别预测和边界框精调

四、语音识别与合成

1. 语音识别 (ASR)

(1) 传统方法: HMM + GMM

- 用隐马尔可夫模型 (HMM) 建模状态转移
- 用高斯混合模型 (GMM) 建模观测概率
- 解码使用 **维特比算法** (Viterbi Algorithm)

(2) 端到端方法: CTC (Connectionist Temporal Classification)

- 允许输入 (音频帧) 与输出 (字符) **不对齐**
- 引入空白符 ϵ , 构建所有合法对齐路径

训练目标: 最大化所有合法路径概率之和

$$P(\mathbf{y} | \mathbf{x}) = \sum_{\pi \in B^{-1}(\mathbf{y})} P(\pi | \mathbf{x})$$

其中 $B(\cdot)$ 为去除重复和空白符的映射函数。

2. 语音合成 (TTS)

(1) 传统流程

- 声学模型: 文本 $\rightarrow$ 声学特征 (如梅尔频谱)
- 声码器 (Vocoder): 声学特征 $\rightarrow$ 波形

(2) FastSpeech (非自回归 TTS)

- 利用 **长度预测器** 控制每个音素的持续时间
- 实现并行生成, 大幅提升速度

例如: 文本 "the cat" $\rightarrow$ 扩展为 `[t, t, t, h, h, e, e, e, ...]`, 再并行合成

- 对齐信息由预训练自回归模型 (如 Transformer TTS) 的 **注意力矩阵** 自动生成

五、科学计算 (AI for Science)

1. 物理信息神经网络 (PINN)

将物理方程作为约束融入神经网络训练。

考虑偏微分方程 (PDE):

$$\begin{cases} \mathcal{N}(t, x; u(t, x; \theta)) = 0, & t \in [0, T], x \in \Omega \\ \mathcal{I}(x; u(0, x; \theta)) = 0, & x \in \Omega \\ \mathcal{B}(t, x; u(t, x; \theta)) = 0, & t \in [0, T], x \in \partial\Omega \end{cases}$$

其中:

- $\mathcal{N}$: 微分算子 (含时间/空间导数)
- $\mathcal{I}$: 初始条件
- $\mathcal{B}$: 边界条件

PINN 总损失函数:

$$\mathcal{L}(\theta) = w_p \mathcal{L}_p(\theta) + w_i \mathcal{L}_i(\theta) + w_b \mathcal{L}_b(\theta) + w_d \mathcal{L}_d(\theta)$$

各项定义:

- PDE 残差损失:**

$$\mathcal{L}_p(\theta) = \frac{1}{N_p} \sum_{i=1}^{N_p} \|\mathcal{N}(t_i, x_i; u(t_i, x_i; \theta))\|^2$$

- 初始条件损失:**

$$\mathcal{L}_i(\theta) = \frac{1}{N_i} \sum_{j=1}^{N_i} \|\mathcal{I}(x_j; u(0, x_j; \theta))\|^2$$

- 边界条件损失:**

$$\mathcal{L}_b(\theta) = \frac{1}{N_b} \sum_{k=1}^{N_b} \|\mathcal{B}(t_k, x_k; u(t_k, x_k; \theta))\|^2$$

- **数据拟合损失**（若有真实观测）：

$$\mathcal{L}_d(\theta) = \frac{1}{N_d} \sum_{l=1}^{N_d} \|u_{\text{true}}(t_l, x_l) - u(t_l, x_l; \theta)\|^2$$

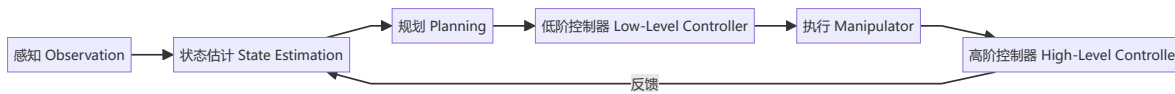
PINN 可在**无标签或少标签**场景下求解 PDE，实现“知识驱动 + 数据驱动”融合。

六、跨媒体智能

- 目标：打通**文本、图像、音频、视频**等异构模态间的语义鸿沟
- 典型任务：
 - 跨媒体检索（图文互搜）
 - 图文生成（Image Captioning / Text-to-Image）
 - 视频问答（Video QA）
 - 音乐驱动视频生成
- 核心挑战：**跨模态对齐** 与 **综合推理**

七、机器人控制

机器人任务执行流程：



- **感知**：通过摄像头、激光雷达、IMU 等传感器获取环境信息
- **状态估计**：融合多源信息，估计自身与环境状态（如 SLAM）
- **规划**：路径规划、任务调度（如 A*、RRT、POMDP）
- **控制**：底层电机控制 + 高层行为协调

八、延伸概念与哲学思考

1. 人工智能效应（AI Effect）

“一旦某件事用 AI 实现了，人们就不再称它为 AI。” —— John McCarthy
→ 技术成熟后被视为“普通计算”

2. 通用目的技术（General Purpose Technology, GPT）

- 特征：普遍适用、持续演进、促进创新互补
- 历史案例：蒸汽机、电力、计算机、互联网
- AI 正成为新一代 GPT，赋能“AI + X”交叉领域

3. “学”与“术”之辨（梁启超）

- 学：发现真理（理论研究）
- 术：致用于世（工程应用）
- 二者相辅相成，推动 AI 从“模拟智能”走向“赋能社会”

九、本章小结

- 人工智能正从“模拟人类智能”转向“**赋能社会**”
- 核心驱动力：**外部需求 > 内部理论**
- 应用领域广泛覆盖：
 - 语言大模型（GPT、BERT）
 - 机器翻译（NMT）
 - 视觉理解（CNN、Faster R-CNN）
 - 语音技术（CTC、FastSpeech）
 - 科学计算（PINN）
 - 跨媒体智能
 - 机器人系统
- 未来方向：**学科交叉融合**（AI + 物理/生物/化学/社科等）

“我们必须知道，我们必将知道。” —— David Hilbert
人工智能的探索永无止境。